

# 无线电干扰源的快速自动定位与清除

## ——覆盖保证、布局下界与在线调度的一体化模型

### 摘 要

本文把“机器狗自动定位并清除无线电干扰源”建模为部分可观测环境下、带覆盖保证的在线搜索-定位-清除问题，按“覆盖保证 → 布局下界 → 在线调度 → 有界误差定位与认证清除”四层求解。全文坚持一条准则：清除率必须为 100%，只有在此前提下才比较时间。

针对问题一，把每个示向度读数转化为两条线性半平面约束，定位区域是有限个前向楔形的交  $P = \bigcap_i W_i$ 。据此设计“线性规划判空与判无界 → 边界线两两求交并过滤 → 极角排序 → 顶点对枚举”的求直径算法，复杂度  $O(m^3 + k^2)$ ，并完整处理空集、无界、线段与单点四类退化。对后半问给出否定回答与紧的定量结论：由 Jung 定理  $D/2 \leq R_* \leq D/\sqrt{3}$ ，故同直径圆一般不能覆盖定位区域；本文由三个真实检测点构造出交集恰为边长 40 m 正三角形的算例， $D = 40.0000$  m 而  $R_* = 23.0940$  m  $> 20$  m。由此确立全文的清除判据： $R_* \leq 20$  m 才可一次认证清除， $D \leq 20\sqrt{3} = 34.641$  m 是其只依赖直径的充分条件。

针对问题二，导出一个不依赖未知源距与未知接收半径的保证接收区域  $C_{\text{safe}}$ ——它是三个半径 1000 m 圆盘之交（见 (12)），并证明其充分性。本文进一步证明  $\sin \psi = \text{dist}(g, L)/\|q - g\|$ ，即横向偏移单独决定交会角的下界，前进距离对此无贡献。在  $C_{\text{safe}}$  上做全域搜索表明，对  $r_1 = 1500$  m 的源，两次测向最好也只能把  $R_*$  降到约 32.5 m，故光学覆盖兜底不可省略，这决定了后两问的算法结构。

针对问题三，证明七站布局（圆心加半径  $r$  的正六边形）的最坏覆盖距离为  $d_{\max} = \max\{r/\sqrt{3}, \sqrt{R^2 + r^2 - \sqrt{3}Rr}\}$ ，可行区间  $r \in [1122.96, 1732.05]$  m；取  $r = 1150$  m 得  $d_{\max} = 988.5114$  m  $< 1000$  m。再由 Bezdek 界  $1.7988 < 1.8$  证明七站是保证发现的最小站数。在线层由“搜索与目标混合贪心调度、有界误差可行域递推、目标圆外切约束与 no\_signal 圆盘排除、最小包围圆认证清除与 25 m 网格兜底、圆心补测精化”构成。官方模拟器演练 14 局、185 个源全部清除，合并 248.3 s/源，距路线下界 6% ~ 9%。

针对问题四，给出方向覆盖的充要条件：位于  $g$  的源对任意未知朝向都能被发现，当且仅当  $g \in \text{conv } \mathcal{N}_\rho(g)$ ，其中  $\mathcal{N}_\rho(g)$  为与  $g$  距离不超过  $\rho$  的测站集。该条件比常用的“剖分边长小于  $\rho$ ”宽松，据此把测站由 49 → 27 → 25 进一步压到 22 站，并用  $h = 1$  m 网格证书（1018.68 万个网格点，最小余量  $1.7331$  m  $> \delta = 0.7071$  m）完成连续域严格验证，同时以三条引理证明 21 站及以下不可行。在线层增加选择性重测、内环目标有界等待、认证条带覆盖（矩形四角到中心不超过 19.9 m）与扫描成本门控。220 例新种子留出验证全部清除（随机集 494.5 → 478.4 s/源）；官方演练 10 局、123 个源全部清除，合并 514.6 s/源，按源数分层后与本地预测的最大偏差仅 23 s/源。

本文进一步给出一个结构性结论：整局总时间主要由与源数无关的固定成本决定

(演练拟合  $T_{Q3} \approx 1886 + 106n$ ,  $T_{Q4} \approx 5765 + 46n$ ), 故“平均定位清除时间”随源数下降主要是分母效应, 正式测试成绩将主要由抽到的源数决定; 所建立的下界模型表明现有方案距理论极限约 8% ~ 10%。

**关键词:** 交会定位; 有界误差可行域; 覆盖保证; 凸包方向覆盖; 在线调度; 最小包围圆

# 一、 问题重述

## 1.1 问题背景

无线电干扰源的定位与清除是频谱管理的核心任务，通常经历初步监测、区域性排查与精准定位三个阶段。前两个阶段可由固定监测站、监测车或无人机完成，可把干扰源圈定到一片区域；但受环境遮挡与低功率辐射的限制，最后的精准定位仍须由技术人员携带便携式测向机徒步完成，效率低且在危险环境中风险大。本题考察的正是这最后一环：用搭载测向机、光学探测仪与激光枪的机器狗，在一个已知的圆形目标区域内自动完成发现一定位一清除的全过程，并使总耗时尽可能短。

目标区域是以原点为心、半径  $R = 1800\text{ m}$  的圆盘，正东为  $x$  轴正向、正北为  $y$  轴正向。区域中干扰源个数未知（第三、四问为  $10 \sim 16$  个），每个源占用  $1 \sim 20$  中互不相同且不随时间变化的一个频道，其位置、有效接收半径（介于  $1000\text{ m}$  与  $1500\text{ m}$  之间）以及（第四问的）定向方向都不可直接读取。机器狗只能通过 `/measure` 与 `/clear` 两条指令与环境交互，这使问题本质上是一个部分可观测的在线决策问题 [10]，而不是已知坐标的路径规划问题。

## 1.2 需要解决的问题

- **问题一（定位区域的直径与覆盖判定）**。给定若干检测点及其对同一干扰源的示向度，要求给出计算多边形定位区域直径的算法，并判定以该直径为直径的圆能否覆盖该区域。其数学本质是角度误差约束下的凸多边形度量问题。
- **问题二（第二检测点的选择）**。已知某全向源在首个检测点处的示向度，要求给出第二个检测点的选择策略及其可行的候选区域。其本质是在“第二点必能收到信号”这一对未知量一致成立的约束下，权衡定位精度与移动代价。
- **问题三（全向源的搜索、定位与清除）**。区域内共有  $10 \sim 16$  个全向源而真实总数未知，机器狗须在保证全部清除的前提下使任务时间最短。其本质是一个带覆盖保证的部分可观测在线搜索与调度问题：核心在于用最小测站集证明搜索完备性，并以在线贪心策略压缩固定与移动成本。
- **问题四（混合全向源与定向源）**。源中同时含有定向源，其定向方向未知、有效覆盖为方向两侧各  $90^\circ$ ，其余条件同问题三。其本质是把问题三的距离覆盖升级为位置与朝向联合的方向覆盖。

### 1.3 关键物理与计时规则

题目与附件给出的规则构成了模型的全部硬约束，汇总于表 1。其中有三条特别影响建模：

1. 测向误差是有界的、确定性的，而不是随机的。题目只保证误差落在  $[-1^\circ, 1^\circ]$  内，且“同一地点重复检测不会改变检测误差”。因此不能假设零均值、独立或高斯分布，也不应使用最小二乘或卡尔曼滤波这类依赖概率模型的估计器；正确的处理是集员（set-membership）定位 [11]。
2. 清除的判据是距离而不是方向。光学精确定位与激光清除只要求清除点与源的距离不超过 20 m，与信号覆盖角度范围无关。这使得“即使再也收不到某定向源的信号，只要还持有它的一次方位约束，仍能用光学覆盖把它清掉”成为可能，是第四问兜底机制的基础。
3. 移动、切频与检测的耗时结构固定。移动 5 m/s、切频 1 s、检测 5 s、清除失败 3 s、清除成功 5 s。这些常数决定了后文的目标函数与成本权衡：一次额外检测（5 ~ 6 s）约相当于 25 ~ 30 m 的移动，因此顺路增加一次检测通常是划算的。

表 1 题目给定的物理规则与计时常数

| 项目     | 规则                                   | 取值                    |
|--------|--------------------------------------|-----------------------|
| 目标区域   | 以原点为心的圆盘，东为 $x$ 正向、北为 $y$ 正向         | $R = 1800 \text{ m}$  |
| 源个数    | 第三、四问，不通过接口返回                        | 10 ~ 16               |
| 频道     | 整数，一源一频道，互不相同且不变                     | 1 ~ 20                |
| 示向度误差  | 全局有界；同点重复检测误差固定                      | $\pm 1^\circ$         |
| 有效接收半径 | 每源一值，未知                              | 1000 ~ 1500 m         |
| 定向覆盖   | 定向方向两侧各 $90^\circ$ ，含边界（闭半平面）        | $180^\circ$           |
| 移动     | 直线，可走出目标区域                           | 5 m/s                 |
| 检测     | 停止移动后调整天线至稳定读数                       | 5 s                   |
| 切换频道   | 任意两频道之间                              | 1 s                   |
| 光学清除   | 距离 $\leq 20 \text{ m}$ 必成功，与辐射方向无关   | 成功 5 s / 失败 3 s       |
| 近场     | 距离 $\leq 5 \text{ m}$ 且在覆盖角内，返回 near | 5 m                   |
| 虚拟时间上限 | 防止死循环                                | 360000 s              |
| 程序运行时间 | 25 分钟窗口与 20 分钟程序时限取早者                | $\leq 1200 \text{ s}$ |

## 二、 模型假设与符号说明

### 2.1 模型假设

本文的全部几何保证都建立在下列假设之上。本文刻意把假设写得窄而明确，以便在第九、节中逐条检查其失效后果。

**假设 1 (静态环境)** 在一次任务过程中，干扰源的位置、频道、有效接收半径与定向方向均不随时间变化；机器狗可沿任意直线自由行走，不考虑障碍、地形与高度差。

**假设 2 (确定性阈值)** 接收（距离不超过  $\rho_i$  且在覆盖角内）、近场（不超过 5m）与清除（不超过 20m）三个判据均为确定性阈值，不存在概率性漏检或误清除；不同干扰源的信号互不影响。

**假设 3 (有界测向误差)** 若在点  $s$  对源  $g$  获得示向度读数  $\theta$ ，则真实方位角  $\theta^{\text{true}}$  满足  $|\theta - \theta^{\text{true}}| \leq \varepsilon$ ，其中题目给出  $\varepsilon = 1^\circ$ 。算法内部统一使用保守界  $\bar{\varepsilon} = 1.005^\circ$ ，以吸收“误差先产生、再按两位小数取整”可能带来的额外偏差。除有界性外，不假设误差的任何分布、独立性或零均值性质。

**假设 4 (信息边界)** 策略只能使用 `/measure` 与 `/clear` 的返回值 (`no_signal`、`near`、`direction` 与示向度、清除成败)，不得读取源的个数、位置、频道集合、接收半径或定向方向。所有本地实验中生成的真值仅用于事后核验，不参与任何决策。

**假设 5 (计时模型)** 虚拟时间严格按附件规则累加：一条 `/measure` 指令的耗时为“移动距离除以 5，加上是否切频对应的 0 或 1 秒，再加 5 秒”；一条 `/clear` 指令的耗时为“移动距离除以 5，再加 3 秒（失败）或 5 秒（成功）”。`/clear` 的频道参数不改变测向机当前频道，`/enter` 与 `/exit` 本身不耗时。

**假设 6 (本地实验生成器，仅用于离线比较)** 本地配对实验中，源数在 10 ~ 16 均匀抽取，位置在圆盘内按面积均匀生成，频道无放回抽样，接收半径在 [1000, 1500] 均匀生成；第四问每个源以 0.5 的概率为定向源且方向均匀。测向误差由“种子、频道与坐标”的哈希确定，使同一地点重复检测得到同一误差。这些分布是本地假设，不是题目事实，官方案例的分布未公开，因此本地均值只用于版本之间的配对比较，不作为对官方成绩的预测。

## 2.2 符号说明

| 符号                               | 意义                                         |
|----------------------------------|--------------------------------------------|
| $\mathcal{D}$                    | 目标区域圆盘，半径 $R = 1800 \text{ m}$ ，圆心在 origin |
| $g_i, c_i, \rho_i, d_i$          | 第 $i$ 个源的位置、频道、有效接收半径、定向方向单位向量             |
| $\rho_{\min}, \rho_{\max}$       | 接收半径的已知界，1000 m 与 1500 m                   |
| $\varepsilon, \bar{\varepsilon}$ | 示向度误差真界 $1^\circ$ 与算法采用的保守界 $1.005^\circ$  |
| $\mathbf{u}(\alpha)$             | 单位向量 $(\cos \alpha, \sin \alpha)^\top$     |
| $[\mathbf{a}, \mathbf{b}]$       | 二维叉积 $a_x b_y - a_y b_x$                   |
| $s_k, \theta_k$                  | 第 $k$ 个检测点及其示向度读数                          |
| $W_k$                            | 由 $(s_k, \theta_k)$ 生成的前向楔形（两条半平面之交）       |
| $P$                              | 定位区域（可行域）， $P = \bigcap_k W_k$ 再与先验约束求交    |
| $D = \text{diam}(P)$             | 定位区域的直径                                    |
| $R_*(P), c_*(P)$                 | $P$ 的最小包围圆半径与圆心                            |
| $\mathcal{S}$                    | 搜索站集合； $ \mathcal{S} $ 为站数                 |
| $\mathcal{N}_\rho(g)$            | 与 $g$ 距离不超过 $\rho$ 的测站之集，见 (29)            |
| $v, t_m, t_s$                    | 速度 5 m/s，检测 5 s，切频 1 s                     |
| $t_f, t_c$                       | 清除失败 3 s，清除成功 5 s                          |
| $r_c, r_n$                       | 清除半径 20 m，近场半径 5 m                         |
| $L, N_m, N_s$                    | 累计移动距离、检测次数、切频次数                           |
| $N_f, N_c$                       | 清除失败次数、成功清除数                               |
| $T, \bar{T}$                     | 整局虚拟总时间；平均定位清除时间 $\bar{T} = T/N_c$         |

全文内部计算一律使用弧度，接口输入输出保持度与米。方位角一律采用题目约定：东为  $0^\circ$ 、逆时针为正、取值范围  $[0^\circ, 360^\circ)$ ，不使用“以北为零、顺时针”的航海约定。

### 三、 问题分析与总体框架

#### 3.1 目标函数的精确形式

按假设 5，把一局任务中的全部动作按类型归并，整局虚拟总时间可以写成一个精确而非近似的线性式：

$$T = \underbrace{\frac{L}{v}}_{\text{移动}} + \underbrace{t_m N_m}_{\text{检测}} + \underbrace{t_s N_s}_{\text{切频}} + \underbrace{t_f N_f}_{\text{光学失败}} + \underbrace{t_c N_c}_{\text{成功清除}} = \frac{L}{5} + 5N_m + N_s + 3N_f + 5N_c. \quad (1)$$

这里  $t_c = 5\text{s}$  已经包含了成功那一次的光学精确定位  $3\text{s}$  与激光清除  $2\text{s}$ ，不能再额外叠加一次  $3\text{s}$ 。题目要求的两个统计量是

$$\text{清除比例} = \frac{N_c}{n}, \quad \bar{T} = \frac{T}{N_c}, \quad (2)$$

其中  $n$  为该局真实源数。于是优化问题为

$$\min_{\pi} \mathbb{E}[T(\pi)] \quad \text{s.t.} \quad N_c(\pi) = n \text{ 以概率 } 1, \quad (3)$$

$\pi$  是一个只依赖历史观测的在线策略（假设 4）。约束写成“以概率 1”而不是“以高概率”，是本文所有布局设计的出发点：本文以清除率为硬约束，时间为目标，不以牺牲覆盖保证换取时间（该取舍的定量代价在第九、节中另行讨论）。

由 (2) 可见， $\bar{T}$  的分母是  $n$ 。下面的第 3.2 节将说明， $T$  的主体是与  $n$  关系较弱的固定成本，因此  $\bar{T}$  随  $n$  下降主要是分母效应。这一点在解释三次正式测试成绩时至关重要。

#### 3.2 时间去哪儿了：固定成本主导

对官方模拟器演练日志按动作类型做分解（第三问 14 局、第四问 10 局，均为当前版本，见第八、节），得到表 2。

表 2 官方演练中虚拟时间的构成（按动作类型累计占比）

| 问题                   | 移动    | 检测    | 切频   | 光学失败 | 激光清除 |
|----------------------|-------|-------|------|------|------|
| 问题三（7 站，14 局，185 源）  | 73.2% | 18.6% | 3.5% | 3.9% | 0.8% |
| 问题四（22 站，10 局，123 源） | 73.4% | 21.2% | 3.9% | 1.1% | 0.4% |

移动占七成以上、检测占两成，二者合计约 92%；真正用于“清除”的时间不到 1%。进一步把总时间对源数做线性回归，得到

$$T_{Q3} \approx 1886 + 106n \text{ (s)}, \quad T_{Q4} \approx 5765 + 46n \text{ (s)}. \quad (4)$$

截距即**固定成本**：巡站路线加上对空频道的排除性扫描。正式测试不告知源数，20 个频道中未被占用的那些必须在所有测站上都测过一遍才能宣告不存在，这部分开销与  $n$  无关。斜率则是每多一个源所需的定位与清除增量。由 (4) 可得

$$\bar{T}_{Q3} \approx \frac{1886}{n} + 106, \quad \bar{T}_{Q4} \approx \frac{5765}{n} + 46, \quad (5)$$

即  $n = 10$  与  $n = 16$  的  $\bar{T}$  可以相差近一倍。据此可以得到两条结论：

1. **优化的主战场是固定成本，即测站布局**。第四问每减少一个测站，约节省 120 s 的空频道扫描加上 140 s 的巡站移动，合计约 260 s；第三问每站约 320 s。相比之下，调度层的精细改进在第五轮实验中总共只换来约 15 s/ 源。
2. **减站受覆盖保证的硬约束**。因此“能减到几站”本身是一个需要证明的几何问题，这正是第六、节与第七、节的核心内容。

### 3.3 四层求解框架

综合上述分析，本文把四个问题统一在同一个框架下（图 1）：

**第一层：覆盖保证（发现）**。预先构造一个测站集  $\mathcal{S}$ ，使得无论源在哪里、朝向如何，都至少存在一个测站能收到它。第三问只需距离覆盖；第四问还需方向覆盖，二者的数学条件截然不同（定理 9）。这一层给出了算法的终止证明：所有测站访问完且已发现目标全部清除，则不可能还有漏网之鱼。

**第二层：布局下界（省时）**。在覆盖保证成立的前提下，测站数与巡回长度决定了固定成本。本文分别证明第三问 7 站为最小（定理 7）、第四问 22 站通过严格网格证书而 21 站不可行（第 7.4 节）。

**第三层：在线调度（降低移动成本）**。把“去下一个测站”与“处理一个已发现目标”放进同一个任务池，按当前移动代价贪心选择，使巡站与定位清除交错执行，避免“先扫完再回头”的长距离折返。

**第四层：有界误差定位与认证清除（保证清除正确性）**。用集员方法维护每个频道的可行域  $P$ ， $P$  始终包含真源；当  $R_*(P) \leq r_c$  时一次认证清除，否则用具有连续覆盖证明的光学点列兜底。第一、二问正是这一层的理论基础。

### 3.4 四个问题之间的关系

表 3 给出四问的输入、决策与关键陷阱。需要强调，第三、四问**不是**已知坐标的旅行商问题，而是部分可观测的在线搜索控制问题：在没有把所有测站走完之前，算法根本不知道还有几个源、在哪里。可以借鉴路径优化的思想，但必须先回答“哪里还可能藏着未发现的源”和“什么时候可以安全地结束”。



【此处插图】图 0：总体求解框架流程图

画法建议：自上而下四个横向色带，分别标注“覆盖保证层 / 布局下界层 / 在线调度层 / 定位与清除层”。左侧主流程为：/enter → 生成测站集 → {任务池决策：最近未访问测站 vs. 最低成本目标} → 到站扫描未清除频道 → 新增/更新可行域 → 补测或精化 → MEC 认证清除或光学覆盖 → 共享测站扫描 → 终止判据（全站访问完毕且目标清空 / 已清除 16 个）→ /exit。右侧用虚线箭头标出每一层对应的定理编号。

图 1 总体求解框架

表 3 四个问题的输入、目标与关键陷阱

| 问题 | 输入与决策           | 目标              | 必须处理的陷阱                                       |
|----|-----------------|-----------------|-----------------------------------------------|
| 一  | 检测点与示向度；求交会可行域  | 算出直径，判断同直径圆能否覆盖 | 是前向楔形（单侧锥）而非双向直线带；空集、无界、线段、单点；直径 $\neq$ 包围圆直径 |
| 二  | 一次全向源示向度；选第二点   | 在移动代价与定位精度之间取舍  | 源距未知；第二点必须收得到信号；不能只追求正交交会                     |
| 三  | 在线选择位置、频道、测向或清除 | 先保证全清，再缩短总时间    | 不知道真实总数；检测不能并行；停止必须有证据                        |
| 四  | 再加入未知源类型与定向方向   | 同上              | no_signal 不能按距离直接排除；边界朝外源需要区域外测站              |

## 四、 问题一：交会定位区域的直径算法与覆盖性判定

### 4.1 从示向度读数到线性约束

设在检测点  $s_k$  处测得某干扰源的示向度为  $\theta_k$ ，误差界为  $\varepsilon$ 。由假设 3，真源  $g$  相对  $s_k$  的真实方位角落在  $[\theta_k - \varepsilon, \theta_k + \varepsilon]$  内，因此  $g$  位于以  $s_k$  为顶点、张角  $2\varepsilon$  的前向楔形

$$W_k = \left\{ x \in \mathbb{R}^2 \mid [\mathbf{u}(\theta_k - \varepsilon), x - s_k] \geq 0, [\mathbf{u}(\theta_k + \varepsilon), x - s_k] \leq 0 \right\} \quad (6)$$

之中，其中  $[\mathbf{a}, \mathbf{b}] = a_x b_y - a_y b_x$ 。定位区域即

$$P = \bigcap_{k=1}^K W_k. \quad (7)$$

**引理 1 (两条不等式已自动选出前向分支)** 当  $2\varepsilon < 180^\circ$  时, (6) 恰好刻画以  $\mathbf{u}(\theta_k)$  为轴、半角为  $\varepsilon$  的单侧锥, 而不包含背向方向。

**证明.** 令  $x - s_k = r \mathbf{u}(\varphi)$  ( $r > 0$ )。两条约束分别化为  $\sin(\varphi - \theta_k + \varepsilon) \geq 0$  与  $\sin(\varphi - \theta_k - \varepsilon) \leq 0$ , 即  $\varphi - \theta_k + \varepsilon \in [0, \pi]$  且  $\varphi - \theta_k - \varepsilon \in [-\pi, 0] \pmod{2\pi}$ , 两者之交为  $\varphi - \theta_k \in [-\varepsilon, \varepsilon]$ 。特别地取  $\varphi = \theta_k + 180^\circ$  时第一式给出  $-\sin \varepsilon < 0$ , 故背向射线被排除。  $\square$

引理 1 并非可有可无的细节: 若把示向度处理成一条“带宽为  $2\varepsilon$  的双向直线带”, 则背向的一整块区域会被错误地保留, 在多站交会时可能产生虚假的交集。

把 (6) 的两条约束写成行向量形式,  $K$  个检测点共给出  $m = 2K$  条线性不等式, 于是

$$P = \{x \in \mathbb{R}^2 : \mathbf{A}x \leq \mathbf{b}\}, \quad \mathbf{A} \in \mathbb{R}^{m \times 2}, \mathbf{b} \in \mathbb{R}^m. \quad (8)$$

$P$  是闭凸集, 但不一定有界, 也可能为空或退化。

## 4.2 直径的计算: 算法与正确性

**定理 1 (直径在顶点上取到)** 设  $P$  为有界闭凸多边形, 顶点集为  $V(P) = \{v_1, \dots, v_k\}$ , 则  $\text{diam}(P) = \max_{i < j} \|v_i - v_j\|$ 。

**证明.**  $\|\cdot\|$  是凸函数, 故对固定的  $y$ ,  $x \mapsto \|x - y\|$  在紧凸集  $P$  上的最大值必在极点处取到; 对  $y$  再用一次同样的论证即可。由 Minkowski 定理, 有界闭凸多边形的极点恰为其顶点。  $\square$

由定理 1, 求直径只需枚举顶点对, 而不必在边界上做连续优化。据此给出算法 1。它的关键不在于求直径这一步本身, 而在于**先把四类情形分清楚, 再计算**: 直接对约束做顶点枚举而不判空、不判无界, 会在读数互相矛盾或测站张角不足时输出毫无意义的数字。

**复杂度.** 判空与判无界共 5 次二维线性规划; 顶点枚举为  $O(m^2)$  对交点、每个交点  $O(m)$  次可行性检验, 合计  $O(m^3)$ ; 直径枚举  $O(k^2)$ 。本题单个源的检测点很少 ( $K \leq 4$ ), 该实现足够; 若  $K$  很大, 可改用半平面交 ( $O(m \log m)$ ) 加旋转卡壳 ( $O(k)$ ), 总复杂度降至  $O(m \log m)$ 。

**四类退化的处理.**  $P$  为空表示读数与误差界不相容 (例如误差界设小了), 此时必须报错并排查角度约定、取整或坐标系, 不能静默地取某个近似交点;  $P$  无界发生在所有楔形的方向锥未能互相封闭时 (典型情形是只有一个检测点, 或多个检测点近似共线且同侧), 此时直径为  $+\infty$ , 用一个人为的大方框去截断再报一个有限直径是错误的;  $P$  为线段或单点时, 算法 1 给出的  $D$  自然退化, 无需特判。

---

**算法 1** 有界误差交会定位区域的直径算法

---

输入：检测点  $s_1, \dots, s_K$ ，示向度  $\theta_1, \dots, \theta_K$ ，误差界  $\epsilon$

输出：定位区域  $P$  的状态（空 / 无界 / 有界）、顶点表与直径  $D$

- 1: 由 (6) 组装  $A \in \mathbb{R}^{2K \times 2}$ ,  $b \in \mathbb{R}^{2K}$
  - 2: 解可行性线性规划  $\min\{0 : Ax \leq b\}$
  - 3: 若不可行则 返回 空集（读数与误差界相互矛盾，应报错而不是输出伪定位点）
  - 4: 判断结束
  - 5: 对  $c \in \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$  循环
  - 6:   解  $\min\{c^\top x : Ax \leq b\}$
  - 7:   若无下界则 返回 无界,  $D = +\infty$
  - 8:   判断结束
  - 9: 循环结束
  - 10:  $V \leftarrow \emptyset$
  - 11: 对每一对约束  $(i, j)$ ,  $i < j$  循环
  - 12:   若  $\det(a_i, a_j) \neq 0$  则
  - 13:     解  $a_i^\top x = b_i, a_j^\top x = b_j$  得交点  $x$
  - 14:     若  $Ax \leq b + \tau \mathbf{1}$  且  $x \notin V$ （按容差去重）则  $V \leftarrow V \cup \{x\}$
  - 15:     判断结束
  - 16:   判断结束
  - 17: 循环结束
  - 18: 以  $V$  的重心为极点，按极角对  $V$  排序，得到凸多边形顶点序列
  - 19:  $D \leftarrow \max_{v_i, v_j \in V} \|v_i - v_j\|$       ▷ 线段与单点自然退化为  $D = \|v_1 - v_2\|$  与  $D = 0$
  - 20: 返回 有界,  $V, D$
- 

### 4.3 以定位区域直径为直径的圆能否覆盖定位区域

**定理 2 (覆盖性的充要条件与紧界)** 设  $P \subset \mathbb{R}^2$  为有界闭凸集,  $D = \text{diam}(P)$ ,  $R_* = R_*(P)$  为最小包围圆半径。则

$$\frac{D}{2} \leq R_* \leq \frac{D}{\sqrt{3}}, \quad (9)$$

右端为 Jung 不等式，两端均可取等。于是

- 存在以  $D$  为直径的圆覆盖  $P$ ，当且仅当  $R_* = D/2$ ；
- 一般情况下只能保证：以  $\frac{2D}{\sqrt{3}} \approx 1.1547D$  为直径的圆必能覆盖  $P$ ，且该系数不可改进。

**证明.** 左端：取  $p, q \in P$  使  $\|p - q\| = D$ ，任何包含  $P$  的圆都包含  $p, q$ ，故其半径不小于  $D/2$ 。右端即平面情形的 Jung 定理。取等： $P$  为线段时  $R_* = D/2$ ； $P$  为边长  $D$  的正三角形时  $R_* = D/\sqrt{3}$ 。第一条由  $R_* \geq D/2$  立得。□

**结论：一般不能覆盖.** 由定理 2，只要  $R_* > D/2$ ，以  $D$  为直径的圆无论圆心放在哪里都无法覆盖  $P$ 。这不是抽象反例，而是可以由真实的检测点与示向度产生的：

例 1 (三站交会给出 Jung 界上的正三角形) 取三个检测点与读数

$$\begin{aligned} s_1 &= (-1100, 0), & \theta_1 &= 1^\circ; \\ s_2 &= (590, -952.6279), & \theta_2 &= 121^\circ; \\ s_3 &= (570, 987.2690), & \theta_3 &= 241^\circ, \end{aligned}$$

误差界  $\varepsilon = 1^\circ$ 。按算法 1 计算,  $P$  为有界三角形, 顶点为  $(0, 0)$ 、 $(40, 0)$ 、 $(20, 34.641016)$ , 即边长 40 m 的正三角形。此时

$$D = 40.000000 \text{ m}, \quad R_* = 23.094011 \text{ m}, \quad \frac{D}{2} = 20 \text{ m},$$

$R_*/D = 0.577350 = 1/\sqrt{3}$ , 恰好取到 Jung 不等式的等号。以  $D$  为直径的圆 (半径 20 m) 不能覆盖该三角形, 缺口为 3.094011 m, 见图 2。

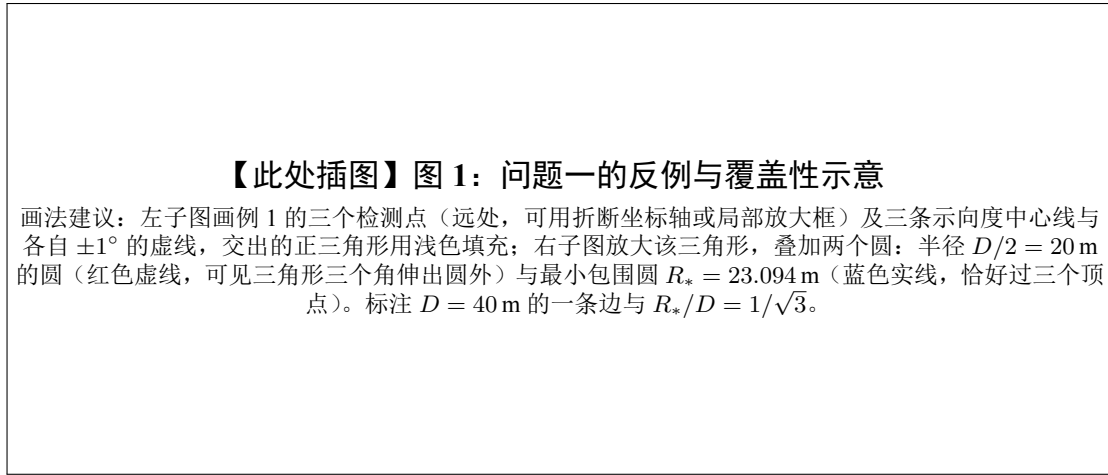


图 2 以定位区域直径为直径的圆一般不能覆盖定位区域

**什么时候可以覆盖: 两站交会的情形。** 需要强调的是, 题目附图所示的两站交会四边形, 当两站到源的距离远大于区域尺度时趋于平行四边形, 此时总能被同直径圆覆盖。设两条楔形的中心线以交会角  $\psi$  相交, 则  $P$  是一个由四条直线围成的“细长四边形”。对平行四边形而言, 长对角线两端是两个锐角顶点, 另外两个顶点的内角为  $180^\circ - \psi (\geq 90^\circ)$ , 因而落在以长对角线为直径的圆内, 即  $R_* = D/2$ 。表 4 给出两个数值算例,  $R_*$  与  $D/2$  在四位小数下完全相同, 验证了这一点。

两例中真源均满足全部线性约束, 即  $g \in P$ , 符合假设 3 下的包含性要求。

#### 4.4 本问结论对后两问的决定性影响

定理 2 直接给出了后两问中“何时可以一次性把源清掉”的正确判据。由于清除成功的条件是清除点与源的距离不超过  $r_c = 20 \text{ m}$ , 而算法只知道  $g \in P$ , 故:

表 4 两站交会的数值算例（真源  $g = (600, 400)$ ， $\varepsilon = 1^\circ$ ，读数含  $+0.6^\circ$  与  $-0.4^\circ$  的偏差）

| $s_1$  | $s_2$     | 交会角 $\psi$    | 顶点数 | $D/\text{m}$ | $R_*/\text{m}$ | $D/2$   | 可否一次认证清除            |
|--------|-----------|---------------|-----|--------------|----------------|---------|---------------------|
| (0, 0) | (800, 0)  | $97.13^\circ$ | 4   | 32.0756      | 16.0378        | 16.0378 | 可 ( $R_* \leq 20$ ) |
| (0, 0) | (1400, 0) | $60.26^\circ$ | 4   | 55.7265      | 27.8632        | 27.8632 | 否，需光学兜底             |

**推论 1 (认证清除判据)** 在  $c_*(P)$  处执行一次 `/clear` 必定成功，当且仅当  $R_*(P) \leq r_c = 20\text{ m}$ 。若仅已知直径，则

$$D \leq \sqrt{3} r_c = 20\sqrt{3} \approx 34.641\text{ m} \quad (10)$$

是  $R_*(P) \leq r_c$  的充分条件，且由例 1 该系数不可放宽到  $2r_c = 40\text{ m}$ 。

因此，工程上常见的“定位区域直径小于  $2 \times 20 = 40\text{ m}$  就能一次清除”的说法是**错误**的：例 1 中  $D = 40\text{ m}$  却存在距圆心  $23.09\text{ m}$  的可能位置。本文后续算法一律以  $R_*(P) \leq 20\text{ m} - 10^{-6}$  作为认证条件（保留浮点余量），而不使用直径判据；只有在需要一个只依赖直径的快速筛选时，才使用 (10)。

## 五、 问题二：第二检测点的选择策略与候选区域

### 5.1 问题的形式化与两个必须同时满足的要求

设首个检测点为  $s_1$ ，测得某全向源的示向度为  $\theta_1$ 。建立以  $s_1$  为原点、 $\mathbf{u}(\theta_1)$  为  $x$  轴正向、 $\mathbf{u}(\theta_1 + 90^\circ)$  为  $y$  轴正向的局部坐标系。记真源在局部坐标下为

$$g = r(\cos \delta, \sin \delta), \quad |\delta| \leq \varepsilon, \quad 0 \leq r \leq \rho \leq \rho_{\max} = 1500\text{ m}, \quad (11)$$

其中  $r$  与该源的真实接收半径  $\rho$  都未知，只知道  $\rho \geq \rho_{\min} = 1000\text{ m}$ ，且因首站收到了信号必有  $r \leq \rho$ 。第二检测点记为  $q = (a, b)$ ，其在原坐标下为  $q = s_1 + a\mathbf{u}(\theta_1) + b\mathbf{u}(\theta_1 + 90^\circ)$ 。

一个“好”的第二检测点必须同时满足两个要求：

**要求 A (可用性)**。在  $q$  处必须确实能收到该源的信号，否则这次移动与检测不能提供任何新的方位约束。由于  $r$  与  $\rho$  未知，这必须是一个不依赖未知量的充分条件。

**要求 B (有效性)**。两条视线的交会必须足够“横”，使得定位区域  $P = W_1 \cap W_2$  足够小。

二者与移动代价  $\|q\|/v$  构成三方权衡。下面分别给出要求 A 的精确刻画（定理 3）与要求 B 的定量刻画（引理 2、(16)），再由二者导出选点策略。

## 5.2 要求 A：保证接收的候选区域

**定理 3 (保证接收的候选区域)** 若第二检测点  $q = (a, b)$  满足

$$C_{\text{safe}} : \begin{cases} a^2 + b^2 \leq \rho_{\min}^2, \\ a^2 + b^2 \leq 2\rho_{\min}(a \cos \varepsilon - |b| \sin \varepsilon), \end{cases} \quad (12)$$

则无论真实源距  $r$  与真实接收半径  $\rho$  取何值，在  $q$  处一定能收到该全向源的信号。等价地，

$$C_{\text{safe}} = B(\mathbf{0}, \rho_{\min}) \cap B(\rho_{\min} \mathbf{u}(\varepsilon), \rho_{\min}) \cap B(\rho_{\min} \mathbf{u}(-\varepsilon), \rho_{\min}), \quad (13)$$

即三个半径为  $\rho_{\min}$  的圆盘之交。

**证明.** 需要证明  $\|q - g\| \leq \rho$ 。因  $\rho \geq \max(\rho_{\min}, r)$ ，只需证  $\|q - g\| \leq \max(\rho_{\min}, r)$ 。

情形一：  $r \leq \rho_{\min}$ . 此时只需  $\|q - g\| \leq \rho_{\min}$ 。函数  $h(r) = \|q - r\mathbf{u}(\delta)\|^2 = r^2 - 2r q^\top \mathbf{u}(\delta) + \|q\|^2$  关于  $r$  是凸二次函数，故在区间  $[0, \rho_{\min}]$  上的最大值必在端点取到。 $r = 0$  时  $h = \|q\|^2 \leq \rho_{\min}^2$ ，即 (12) 第一式； $r = \rho_{\min}$  时要求  $\|q - \rho_{\min} \mathbf{u}(\delta)\| \leq \rho_{\min}$ ，展开即  $\|q\|^2 \leq 2\rho_{\min} q^\top \mathbf{u}(\delta)$ 。由于  $|\delta| \leq \varepsilon$ ， $q^\top \mathbf{u}(\delta) = a \cos \delta + b \sin \delta \geq a \cos \varepsilon - |b| \sin \varepsilon$ ，于是 (12) 第二式是对所有允许的  $\delta$  成立的充分条件。

情形二：  $r > \rho_{\min}$ . 此时  $\rho \geq r$ ，只需  $\|q - g\| \leq r$ ，即  $\|q\|^2 \leq 2r q^\top \mathbf{u}(\delta)$ 。由第二式可知  $q^\top \mathbf{u}(\delta) > 0$ ，故右端关于  $r$  单调增，最紧的情形是  $r = \rho_{\min}$ ，回到第二式。

最后， $\|q\|^2 \leq 2\rho_{\min} q^\top \mathbf{u}(\pm \varepsilon)$  等价于  $\|q - \rho_{\min} \mathbf{u}(\pm \varepsilon)\| \leq \rho_{\min}$ ，而第二式恰为  $\delta = +\varepsilon$  与  $\delta = -\varepsilon$  两种情形同时成立，故得 (13)。□

**推论 2 (候选区域的极值点)**  $C_{\text{safe}}$  中横向偏移的最大值为

$$|b|_{\max} = \rho_{\min} \sin(60^\circ - \varepsilon) = 1000 \sin 59^\circ = 857.167 \text{ m}, \quad (14)$$

在  $a = \rho_{\min} \cos(60^\circ - \varepsilon) = 515.038 \text{ m}$  处取到；前向偏移的最大值为  $a_{\max} = \rho_{\min} = 1000 \text{ m}$  (此时  $b = 0$ )。

**证明.**  $B(\mathbf{0}, \rho_{\min})$  与  $B(\rho_{\min} \mathbf{u}(-\varepsilon), \rho_{\min})$  是两个半径相等、圆心相距  $\rho_{\min}$  的圆盘，其边界交点位于  $\rho_{\min} \mathbf{u}(-\varepsilon \pm 60^\circ)$ ，上方交点即  $\rho_{\min} \mathbf{u}(60^\circ - \varepsilon)$ ，代入即得。□

图 3 给出了  $C_{\text{safe}}$  的形状及其与定位精度、移动代价的关系。

**关于候选区域的两点说明.** 其一， $C_{\text{safe}}$  是充分条件而非充要条件：实际可用的点可能更多（例如源距很近时），但那需要用到未知的  $r$  与  $\rho$ ，不能作为策略依据。其二， $C_{\text{safe}}$  允许  $q$  落在 1800 m 目标圆之外，这是合法的——附件明确允许机器狗走出目标区域。

**【此处插图】图 2：第二检测点的候选区域  $C_{\text{safe}}$**

画法建议：在局部坐标系中画三个半径 1000 m 的圆（圆心分别在原点、 $1000\mathbf{u}(1^\circ)$ 、 $1000\mathbf{u}(-1^\circ)$ ），其公共部分填色即  $C_{\text{safe}}$ ；标出极值点  $(515.0, \pm 857.2)$  与  $(1000, 0)$ ；叠加首测楔形（ $\pm 1^\circ$  的窄扇形）；再用等值线叠加“ $r_1 = 1500$  时的  $R_*$  一阶估计”，可直观看到  $R_*$  随  $|b|$  增大而下降、随移动距离增大而代价上升；最后标出本文实际采用的  $(200, 100)$  与历史基线  $(650, 250)$  两个点。

图 3 保证接收的第二检测点候选区域及其等代价线

### 5.3 要求 B：交会角与定位区域尺度

**引理 2 (交会角由横向偏移单独决定)** 设  $\psi$  为两条视线在真源  $g$  处的夹角， $L$  为过  $s_1$  与  $g$  的直线，则

$$\sin \psi = \frac{\text{dist}(q, L)}{\|q - g\|}, \quad \text{dist}(q, L) = |b \cos \delta - a \sin \delta| \geq |b| \cos \varepsilon - a \sin \varepsilon. \quad (15)$$

**证明.** 在三角形  $s_1 g q$  中， $g$  处的内角为  $\psi$ （或其补角，正弦相同）。由正弦定理  $\|q - s_1\| / \sin \psi = \|q - g\| / \sin \angle(s_1)$ ，而  $\text{dist}(q, L) = \|q - s_1\| \sin \angle(s_1)$ ，整理即得。第二式由  $L$  与局部  $x$  轴夹角为  $\delta$ 、 $|\delta| \leq \varepsilon$  直接算出。□

引理 2 给出了本问最重要的结构性结论：**交会角的下界只由横向偏移  $|b|$ （相对于第二段视距  $r_2 = \|q - g\|$ ）决定，与前进距离  $a$  无关。**“沿着示向度方向一直往前走”对改善交会角毫无帮助，只能缩短  $r_2$ 。

在  $\varepsilon$  为小角的一阶近似下，两条楔形在  $g$  附近可视为宽度  $w_1 = 2r_1 \tan \varepsilon$ 、 $w_2 = 2r_2 \tan \varepsilon$  的条带（ $r_1 = \|g - s_1\|$ ），其交为平行四边形，长对角线与最小包围圆半径为

$$D \approx \frac{\sqrt{w_1^2 + w_2^2 + 2w_1 w_2 |\cos \psi|}}{\sin \psi}, \quad R_* = \frac{D}{2} \lesssim \frac{(r_1 + r_2) r_2 \tan \varepsilon}{2 \text{dist}(q, L)}. \quad (16)$$

（平行四边形满足  $R_* = D/2$ ，与表 4 的精确计算一致。）(16) 是一阶估计：由于楔形宽度沿距离线性增长，精确枚举得到的直径略大于该式，因此下文的定量结论一律以精确枚举为准，(16) 只用于解释趋势。

### 5.4 两次测向不足以保证认证清除

**定理 4 (两测向的能力上限)** 在  $C_{\text{safe}}$  内任取第二检测点，当  $r_1$  接近  $\rho_{\text{max}} = 1500$  m 时，均无法保证  $R_*(P) \leq r_c = 20$  m。按 (16) 在  $C_{\text{safe}}$  上做全域搜索， $r_1 = 1500$  m 时可达到的

最小  $R_*$  约为 32.5 m (取  $q \approx (837, 547)$ , 此时  $\|q\| \approx 1000$  m, 移动即需 200 s), 仍显著大于 20 m。

这一结论有三个直接后果, 贯穿第三、四问:

1. **光学覆盖兜底不可省略.** 任何“先两次测向、再一次清除”的流水线都是不完备的; 必须保留一个对整个可行域  $P$  的、有连续覆盖证明的清除点列。
2. **不能盲目追求正交交会.** 既然远距离源本来就注定要进入光学覆盖, 为了把  $R_*$  从 300 m 降到 40 m 而多走 800 m (160 s) 常常是不划算的——省下的光学扫描未必抵得上多走的路。
3. **近距离源应优先获得第二次测向.** 由 (16),  $R_*$  近似正比于  $r_1 r_2$ , 因此对近源而言两次测向往往足以认证。表 5 给出以 (200, 100) 为第二点时的一阶估计:  $r_1 \lesssim 400$  m 的源基本可以一次认证清除, 更远的则需进入光学覆盖。

表 5 不同候选第二点的定位效果 (一阶估计  $R_*$ , 单位 m) 与移动代价

| 候选点 $(a, b)$            | 移动距离/m | 移动时间/s | $r_1 = 500$ | $r_1 = 1000$ | $r_1 = 1500$ |
|-------------------------|--------|--------|-------------|--------------|--------------|
| (200, 100)              | 223.6  | 44.7   | 22.6        | 127.7        | 320.7        |
| (250, 100)              | 269.3  | 53.9   | 18.2        | 116.6        | 302.9        |
| (300, 300)              | 424.3  | 84.9   | 9.1         | 39.2         | 99.0         |
| (500, 250)              | 559.0  | 111.8  | 6.6         | 30.6         | 91.5         |
| (650, 250)              | 696.4  | 139.3  | 8.1         | 21.6         | 74.2         |
| (750, 450)              | 874.6  | 174.9  | 10.2        | 15.2         | 40.5         |
| (515, 857) <sup>†</sup> | 999.8  | 200.0  | 11.9        | 20.0         | 37.5         |

<sup>†</sup> 为推论 2 给出的最大横向偏移点。所有候选点均落在  $C_{\text{safe}}$  内。

## 5.5 精确枚举验证与选点策略

为避免只依赖一阶近似, 本文对 12 个候选点做了精确枚举: 源距取 21 个值、首测偏差取 3 个方向、第二测偏差取 3 个方向, 共 189 个组合, 逐一按算法 1 计算精确的交会多边形与其直径, 取样本内最坏值, 结果见表 6。

数据印证了引理 2: 固定  $a$  时, 最坏直径随  $b$  增大而迅速下降 (250  $\rightarrow$  450 的侧移可使直径减半); 而固定  $b$  时, 增大  $a$  的收益明显更小。同时, 最好的候选 (750, 450) 的最坏直径仍有 138.06 m, 远大于  $\sqrt{3}r_c = 34.64$  m, 再次印证定理 4。

**选点策略.** 综上, 第二检测点的选择应当按下列顺序:

1. **可行性优先:**  $q \in C_{\text{safe}}$  ((12)), 保证一定收得到信号;
2. **左右镜像取近:**  $b$  取正负两个镜像位置中综合代价更小的一侧;



表 6 12 个候选第二点的精确枚举结果（每点 189 个组合， $\varepsilon = 1^\circ$ ）

| 前进 $a/\text{m}$ | 侧移 $b/\text{m}$ | 移动/s   | 最坏直径/m | 前进 $a/\text{m}$ | 侧移 $b/\text{m}$ | 移动/s   | 最坏直径/m |
|-----------------|-----------------|--------|--------|-----------------|-----------------|--------|--------|
| 250             | 100             | 53.85  | 631.03 | 650             | 100             | 131.53 | 414.19 |
| 250             | 250             | 70.71  | 361.97 | 650             | 250             | 139.28 | 230.30 |
| 250             | 450             | 102.96 | 240.56 | 650             | 450             | 158.11 | 156.35 |
| 500             | 100             | 101.98 | 494.05 | 750             | 100             | 151.33 | 362.11 |
| 500             | 250             | 111.80 | 277.46 | 750             | 250             | 158.11 | 200.52 |
| 500             | 450             | 134.54 | 185.97 | 750             | 450             | 174.93 | 138.06 |

### 3. 以“完成任务所需时间”而非“区域面积”为目标：最小化

$$J(q) = \underbrace{\frac{\|q - p_{\text{cur}}\|}{v}}_{\text{走到第二点}} + \underbrace{t_m + t_s}_{\text{检测与切频}} + \underbrace{\hat{C}_{\text{clear}}(P(q))}_{\text{此后清除该源的估计代价}}, \quad (17)$$

其中  $\hat{C}_{\text{clear}}$  为“若  $R_* \leq r_c$  则走到圆心清除，否则按光学覆盖点列走一遍”的保守估计。

**单目标最优不等于整局最优。** 本文在第三、四问中实际采用的是较短的  $(a, b) = (200, \pm 100)$ ，而不是表 6 中单目标意义下更优的  $(750, 450)$ 。原因有三：其一，由定理 4，远距离源无论如何都要走光学覆盖，把  $R_*$  从 320 m 降到 40 m 换来的扫描节省，抵不过多走 650 m（130 s）的代价；其二，多目标场景中同一次移动可以为多个频道提供观测（共享测站，见第 6.5 节），第二测点不再需要为单个目标“专程”远行；其三，走远之后机器狗离主巡站路线更远，会带来后续折返。

本文用固定种子的配对实验验证了这一判断：在  $\{100, 150, 200, 250, 300\} \times \{50, 100, 150, 200\}$  的离散集合上按信息增益与移动代价自适应选点，与固定  $(200, 100)$  相比，第三问  $263.920 \rightarrow 265.518 \text{ s/源}$ 、第四问  $505.344 \rightarrow 508.205 \text{ s/源}$ ，两问均值与最坏值均无改善（清除率均为 100%）。因此正式方案保留固定的  $(200, \pm 100)$ ，并把“要不要再测一次”的判断交给第 6.4 节的补测精化与第 7.5 节的扫描成本门控。

## 六、 问题三：全向干扰源的自动搜索、定位与清除

### 6.1 三层结构与终止证明

第三问的难点不在“怎样清除一个已知的源”，而在“怎样证明已经没有源了”。本文把策略分成发现—定位—清除三层，其中发现层承担终止证明的责任。

**定义 1 (距离覆盖测站集)** 称测站集  $\mathcal{S}$  对目标区域  $\mathcal{D}$  具有距离覆盖，若

$$d_{\max}(\mathcal{S}) := \max_{x \in \mathcal{D}} \min_{s \in \mathcal{S}} \|x - s\| \leq \rho_{\min} = 1000 \text{ m}. \quad (18)$$

**定理 5 (终止的正确性)** 设  $\mathcal{S}$  具有距离覆盖。若算法在每个  $s \in \mathcal{S}$  处都对每个尚未成功清除的频道检测过一次，并清除了由此发现的所有目标，则  $\mathcal{D}$  内不存在未被清除的全向干扰源。

**证明.** 反设频道  $c$  上还存在未清除源  $g \in \mathcal{D}$ 。由 (18) 存在  $s \in \mathcal{S}$  使  $\|s - g\| \leq 1000 \leq \rho$ 。全向源覆盖角为  $360^\circ$ ，故在  $s$  处检测频道  $c$  必定返回 `direction` 或 `near`，于是  $g$  会被发现并进入目标列表，与“所有发现的目标均已清除”矛盾。  $\square$

定理 5 说明：合法的停止理由只有两个——**(i)** 覆盖测站全部访问完毕且目标列表清空；**(ii)** 已成功清除 16 个源，达到题设数量上界。“连续若干次没有收到信号”“已经清了 10 个”“20 个频道各扫过一遍”都不是有效的终止证明。

## 6.2 发现层：七站布局及其最优性

### 6.2.1 最坏覆盖距离的解析式

由第 3.2 节，固定成本与测站数成正比，因此应在 (18) 的约束下尽量减少站数、缩短巡回。本文采用“圆心 + 半径  $r$  的正六边形”共 7 站的布局

$$\mathcal{S}_7 = \{0\} \cup \{r \mathbf{u}(60^\circ k) : k = 0, 1, \dots, 5\}. \quad (19)$$

**定理 6 (七站布局的最坏覆盖距离)** 对  $0 < r \leq \sqrt{3}R$ ,

$$d_{\max}(\mathcal{S}_7) = \max \left\{ \frac{r}{\sqrt{3}}, \sqrt{R^2 + r^2 - \sqrt{3}Rr} \right\}. \quad (20)$$

**证明.** 设  $x \in \mathcal{D}$ ,  $t = \|x\|$ 。六个环点均匀分布，故存在一个环点  $s_k$  使  $x$  与  $s_k$  的极角差  $\phi$  满足  $|\phi| \leq 30^\circ$ ，于是  $\|x - s_k\| = \sqrt{t^2 + r^2 - 2tr \cos \phi} \leq \sqrt{t^2 + r^2 - \sqrt{3}tr} =: g(t)$ 。又圆心站给出距离  $t$ ，故最近站距离不超过  $f(t) = \min\{t, g(t)\}$ 。

比较两支： $t \leq g(t) \iff t^2 \leq t^2 + r^2 - \sqrt{3}tr \iff t \leq r/\sqrt{3}$ 。因此  $f(t) = t$  于  $[0, r/\sqrt{3}]$ ,  $f(t) = g(t)$  于  $[r/\sqrt{3}, R]$ 。前段最大值为  $r/\sqrt{3}$ 。后段  $g$  是关于  $t$  的二次根式， $g'(t) \propto 2t - \sqrt{3}r$ ，在  $t = \sqrt{3}r/2$  取极小，故在闭区间  $[r/\sqrt{3}, R]$  上的最大值在端点取到，为  $\max\{g(r/\sqrt{3}), g(R)\} = \max\{r/\sqrt{3}, \sqrt{R^2 + r^2 - \sqrt{3}Rr}\}$ 。合并即得 (20)。上界可达： $x$  取  $t = r/\sqrt{3}$  且极角恰在两环点之间（Voronoi 三重点），或  $x$  取目标圆边界上两环点角平分方向的点。  $\square$

**推论 3 (环半径的可行区间与最优值)** 取  $R = 1800 \text{ m}$ 、 $\rho_{\min} = 1000 \text{ m}$ ， $d_{\max} \leq \rho_{\min}$  当且仅当

$$\begin{aligned} r &\in [r_-, r_+], \\ r_- &= \frac{\sqrt{3}R - \sqrt{3R^2 - 4(R^2 - \rho_{\min}^2)}}{2} = 1122.956 \text{ m}, \\ r_+ &= \sqrt{3}\rho_{\min} = 1732.051 \text{ m}. \end{aligned} \quad (21)$$

$d_{\max}(r)$  在  $r^* = \frac{\sqrt{3}}{2}R = 1558.846 \text{ m}$  处取最小值  $R/2 = 900 \text{ m}$ 。

为什么不取覆盖最优的  $r^*$ 。覆盖只是约束而非目标。 $r$  越小，六个环点越靠内，巡站路线越短，而且发现目标后的第二测点、补测点与清除点也都更靠近主路线。因此本文在可行区间内取靠近下界的  $r = 1150 \text{ m}$ ，此时

$$d_{\max} = \sqrt{1800^2 + 1150^2 - \sqrt{3} \times 1800 \times 1150} = 988.5114 \text{ m} < 1000 \text{ m}, \quad (22)$$

覆盖余量  $11.4886 \text{ m}$ 。5 m 网格加边界采样的数值核对（61445 个位置）与该解析值一致。程序在启动时用 (20) 重新校验，不满足覆盖条件即拒绝运行，避免误改参数导致保证失效。

图 4 给出了该布局的覆盖示意与  $d_{\max}(r)$  曲线。实验上， $r$  取 1250, 1300, ..., 1450 的扫描全部慢于 1150；而  $r < 1122.956$  会直接破坏覆盖保证。也就是说， $r = 1150 \text{ m}$  已经贴着可行下界，这一维参数上不再有可挖掘的空间。

### 6.2.2 七站是保证发现的最小站数

**定理 7 (站数下界)** 在  $R = 1800 \text{ m}$ 、 $\rho_{\min} = 1000 \text{ m}$  的条件下，任何具有距离覆盖的测站集至少包含 7 个站；(19) 取  $r \in [r_-, r_+]$  时达到该下界。

**证明.** (18) 等价于用  $|S|$  个半径  $\rho_{\min}$  的闭圆盘覆盖半径  $R$  的圆盘，即覆盖半径比为  $R/\rho_{\min} = 1.8$ 。Bezdek 证明了 6 个单位圆盘所能覆盖的圆的最大半径为  $1.7988... < 1.8[1, 2, 12]$ ，故  $|S| \leq 6$  不可能；而  $\mathcal{S}_7$  在  $r \in [r_-, r_+]$  时满足覆盖，故 7 可达。□

为了从反面确认这一结论的紧性（并检验“少一站能省多少”），本文另外实现了一个不保证覆盖的六站实验入口，结果见表 7。

表 7 六站布局实验（独立入口，默认方案未采用）

| 布局          | 最坏未覆盖距离/m  | 单源漏检概率                     | 120 例均值 s/源 | 与 7 站配对差 | 更快局数          |
|-------------|------------|----------------------------|-------------|----------|---------------|
| 7 站（默认）     | 988.5（有保证） | 0                          | 258.6       | —        | —             |
| 6 站环 960 m  | 1102.0     | $\approx 4 \times 10^{-4}$ | 256.1       | -2.5     | 69/120（漏 1 源） |
| 6 站环 1000 m | 1062.9     | $\approx 2 \times 10^{-4}$ | 257.2       | -1.4     | 61/120（无漏）    |
| 6 站环 1040 m | 1043.5     | $\approx 8 \times 10^{-5}$ | 257.9       | -0.7     | 63/120（无漏）    |

用 Nelder–Mead 做过六站极小极大优化，最坏未覆盖距离最低只能压到约 1003 m（接近理论值 1000.7 m），仍然超过 1000 m。更重要的是：去掉一个站只节省约 6 s/ 源的扫描，却因为失去圆心站而使首次发现更远、清除路程更长，移动增加约 4 s/ 源，净收益仅 1 ~ 2 s/ 源，代价是引入约万分之二的单源漏检概率。在“清除率是硬约束”的准则下这一交换不可接受，因此正式方案坚持七站。这表明第三问的发现层已接近其几何下界。

**【此处插图】图 3：第三问七站布局与覆盖验证**

画法建议：左子图画  $R = 1800\text{ m}$  目标圆、7 个测站（圆心 + 环半径  $1150\text{ m}$  的六点）与它们的 Voronoi 剖分，每站叠加半径  $1000\text{ m}$  的接收圆（半透明），可见并集完整覆盖目标圆；标出两个最坏点（ $r/\sqrt{3} = 663.95\text{ m}$  的 Voronoi 三重点与边界上  $988.51\text{ m}$  的点）。右子图画  $d_{\max}(r)$  随环半径  $r$  的变化曲线（(20)），横轴  $r \in [900, 1800]$ ，标出可行区间  $[1122.96, 1732.05]$ 、最优值  $r^* = 1558.85$ 、以及实际取值  $r = 1150$  及其余量  $11.49\text{ m}$ 。

图 4 七站布局的覆盖证明与环半径的可行区间

### 6.3 定位层：有界误差可行域

对每个已发现频道  $c$  维护一个凸可行域  $P_c$ ，其不变量是

$$(\text{包含性不变量}) \quad g_c \in P_c \text{ 恒成立.} \quad (23)$$

(i) 初始化. 首次收到 `direction` 时，以检测点  $p$  为顶点构造包含真实扇区的三角形外包

$$P_c^{(0)} = \text{conv}\{p, p + \rho_{\max} \mathbf{u}(\theta) - w\mathbf{n}, p + \rho_{\max} \mathbf{u}(\theta) + w\mathbf{n}\}, \quad w = \rho_{\max} \tan \bar{\varepsilon}, \quad \mathbf{n} \perp \mathbf{u}(\theta). \quad (24)$$

它外包了半径  $\rho_{\max}$ 、半角  $\bar{\varepsilon}$  的扇形，因此无需猜测源距。

(ii) 新方位裁剪. 每获得一条新的有效示向度，按 (6) 生成两条半平面并求交： $P_c \leftarrow P_c \cap W$ 。若结果为空，说明模型假设与观测矛盾，程序报错并保留日志，而不是取一个近似点继续。

(iii) 目标圆外包约束. 所有源都在  $\mathcal{D}$  内，故可以再交上目标圆。为保持多边形表示，本文用 48 条支撑半平面  $\mathbf{n}_j^\top \mathbf{x} \leq R + 10^{-7}$  ( $\mathbf{n}_j$  均匀取 48 个方向) 作为圆的外切近似。这里必须是外切而不是内接：内接多边形会把靠近圆边界的一圈真实位置切掉，可能使边界源脱离  $P_c$ ，破坏 (23)。

(iv) 无信号的距离排除（仅第三问）。这是第三问相对第四问独有的信息优势。

**定理 8 (无信号排除)** 第三问中所有源都是全向源。若在点  $s$  检测频道  $c$  返回 `no_signal`，而频道  $c$  上仍存在未清除的源  $g_c$ ，则  $\|s - g_c\| > \rho_c \geq \rho_{\min}$ ，即  $g_c \notin B(s, \rho_{\min})$ 。

于是可令  $P_c \leftarrow \text{conv}(P_c \setminus B^\circ(s, \rho_{\min}))$ 。注意  $P_c \setminus B^\circ$  一般非凸，本文取其凸包作为保守外近似：这可能重新包含一小部分已被排除的区域（约束变松），但绝不会误删真源。实现中排除半径取  $\rho_{\min} - 10^{-4}$ ，为浮点计算留余量。

**一次必须记录的失败。** 在圆盘排除约束的首轮验证中，曾有一例的几何证书失效：该局虽然最终清除了全部源，程序内部却报出“可行域排除了真源”。根因是：重复施加圆盘排除时，一个理论上位于边界的交点被浮点判为落在圆盘内部，相应二次方程的根又略微超出线段参数范围，导致边界顶点被错误丢弃，多次裁剪后可行域出现了不保守的收缩。修复措施为：用平方距离容差保留边界顶点、对接近端点的根做范围容许与夹取、点数不足或凸包退化时保留输入多边形，并增加重复约束的回归测试。该案例的种子、报错与修复前后对比见附录 C。

这次失败给出一条重要教训，也是本文的一个方法论主张：“**结果全部清除**”不能替代对中间几何不变量的检查。所有实验都必须逐条核验可行多边形与包围圆是否真的包含真源。图 5 展示了可行域在上述四类约束下逐步收缩直至可以认证清除的完整过程。

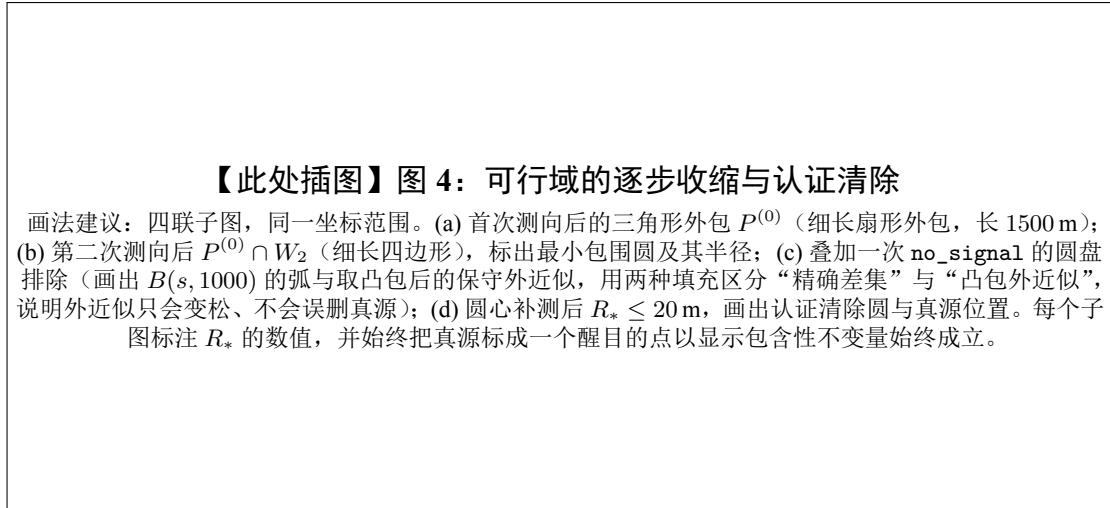


图 5 有界误差可行域的收缩过程

## 6.4 清除层：认证清除、光学兜底与补测精化

(i) **认证清除。** 由推论 1，当  $R_*(P_c) \leq r_c - 10^{-6}$  时，在最小包围圆圆心  $c_*(P_c)$  处执行一次 `/clear` 必定成功。最小包围圆按“1、2 或 3 个支撑顶点”穷举候选圆心并逐一校验，凸多边形只需检查顶点。

(ii) 光学覆盖兜底. 若  $R_* > r_c$ , 用边长  $h_{\text{opt}} = 25 \text{ m}$  的方格覆盖  $P_c$ , 保留所有与  $P_c$  相交的格, 在各格中心依次尝试清除。覆盖正确性来自

$$\frac{h_{\text{opt}}}{\sqrt{2}} = \frac{25}{\sqrt{2}} = 17.677670 \text{ m} < 20 \text{ m} = r_c, \quad (25)$$

即格内任意点到格心的距离都在清除半径内; 由于真源必在  $P_c$  内、 $P_c$  被这些格覆盖, 故必有一格成功。遍历采用按列蛇形路径, 并根据当前位置选择从哪一端进入。

(iii) 补测精化. 认证清除与光学兜底之间存在一个过渡区间:  $R_*$  只比  $20 \text{ m}$  略大 (例如  $60 \text{ m}$ ), 却可能需要在上百米的网格上逐点尝试清除。为此加入圆心补测: 当  $r_c < R_* \leq 300 \text{ m}$  时, 先走到  $c_*(P_c)$  再测一次——该移动与后续清除路径高度重合, 因此边际成本较低; 若仍  $> 20 \text{ m}$ , 则沿新方位的垂线方向横移  $60 \text{ m}$  再补测一次; 此后才进入光学网格。100 例留出验证: 平均  $262.6 \rightarrow 260.0 \text{ s/源}$ , 每源失败清除次数由  $3.00$  次降到  $0.38$  次,  $64$  例更快、 $36$  例更慢、最大单例退步  $30.9 \text{ s/源}$ 。

## 6.5 调度层: 搜索与目标处理的混合贪心

早期版本采用“到一个站  $\rightarrow$  扫完  $\rightarrow$  处理本站发现的全部目标  $\rightarrow$  再去下一站”的分批流程, 会产生大量折返。当前版本把搜索任务与目标任务放进同一个任务池, 每一步比较:

- 去最近未访问测站的代价:  $\|s^* - p_{\text{cur}}\|$ ;
- 处理待清除目标  $c$  的代价:

$$J(c) = \begin{cases} \|q_c - p_{\text{cur}}\| + \|q_c - c_*(P_c)\|, & \text{还需补一条方位,} \\ \|x_c - p_{\text{cur}}\|, & \text{已可直接处理,} \end{cases} \quad (26)$$

其中  $q_c$  为第二测点,  $x_c$  为 **near** 点或最小包围圆圆心  $c_*(P_c)$ ; 取二者中较小者执行。这样机器狗可以“顺路”经过新的测站, 也可以在靠近时提前把近处目标处理掉。目标状态跨测站保留, 不再每到一站就清空。

**共享测站 (shared scan)**. 一次移动应当尽量服务多个目标。到达第二测点或完成一次清除后, 算法检查其他待处理频道: 若当前位置对该频道确有观测价值, 就顺便补测一次。筛选条件见表 8, 其作用是防止共享测量无限增加——它们是选择动作的启发式, 不提供额外的接收保证, 也不参与任何正确性论证。

## 6.6 算法总览

把上述四层合在一起, 得到算法 2。

表 8 共享测站的筛选条件

| 条件                      | 阈值                    |
|-------------------------|-----------------------|
| 与该目标已有采样点的最小距离          | $\geq 100 \text{ m}$  |
| 目标估计中心到当前位置的距离          | $\leq 1300 \text{ m}$ |
| 估计交会角的正弦                | $\geq 0.12$           |
| 每个目标的额外共享检测次数           | $\leq 4$              |
| 已有有效方位数                 | $< 3$ 时才继续            |
| 已清除或已获得 <b>near</b> 的目标 | 跳过                    |

**算法 2** 第三问：七站覆盖 + 混合贪心的在线搜索、定位与清除

输入：目标圆半径  $R$ ，接收半径下界  $\rho_{\min}$ ，误差界  $\bar{\varepsilon}$ ，环半径  $r = 1150 \text{ m}$

输出：全部干扰源被清除

```

1: 调用 /enter;  $p_{\text{cur}} \leftarrow (0, 0)$ ，当前频道  $\leftarrow 1$ ， $\mathcal{C}_{\text{done}} \leftarrow \emptyset$ ，轨迹表  $\mathcal{Q} \leftarrow \emptyset$ 
2: 由 (19) 生成  $\mathcal{S}_7$  并用 (20) 校验覆盖；不满足则拒绝运行
3: 当  $\mathcal{S}_7 \neq \emptyset$  或  $\mathcal{Q}$  中仍有未清除频道时循环
4:    $c^* \leftarrow \arg \min_{c \in \mathcal{Q} \setminus \mathcal{C}_{\text{done}}} J(c)$  ▷ (26)
5:    $s^* \leftarrow \arg \min_{s \in \mathcal{S}_7} \|s - p_{\text{cur}}\|$ 
6:   若  $\|s^* - p_{\text{cur}}\| < J(c^*)$  则 ▷ 搜索更划算
7:   走到  $s^*$  并从  $\mathcal{S}_7$  移除；对每个未清除频道  $c$  依次检测（当前频道优先，其余
   升序）
8:   若  $c \in \mathcal{Q}$  则更新  $P_c$ ，否则在收到信号时新建轨迹并施加定理 8 与目标圆约束
9:   否则 ▷ 处理目标更划算
10:   若  $c^*$  尚需补一条方位则
11:   走到第二测点  $q_{c^*} = \text{origin} + 200 \mathbf{u}(\theta) \pm 100 \mathbf{n}$ （取综合代价小的一侧）并
   检测，更新  $P_{c^*}$ 
12:   共享测站扫描（排除  $c^*$ ）
13:   判断结束
14:   圆心补测精化（第 6.4 节 (iii)）
15:   若  $R_*(P_{c^*}) \leq r_c - 10^{-6}$  则在  $c_*(P_{c^*})$  处清除
16:   否则 按 25 m 方格蛇形遍历  $P_{c^*}$  逐格尝试清除
17:   判断结束
18:   共享测站扫描
19:   判断结束
20:   若  $|\mathcal{C}_{\text{done}}| = 16$  则 break ▷ 题设数量上界
21:   判断结束
22: 循环结束
23: 调用 /exit

```

**虚拟时间上界。** 所有行动点都在以原点为心、2000 m 为半径的圆内，任意两点距离不超过 4000 m。7 个测站每站至多 20 次含切频的检测（120 s）；至多 16 个目标，每个目标至多两段全局跳转、一条保守长度 6500 m 的光学路线、若干次检测与至多 183 次清除

尝试。于是

$$T \leq 7\left(\frac{4000}{5} + 120\right) + 16\left[\frac{2 \times 4000 + 6500}{5} + 6 \times 6 + 183 \times 3 + 5\right] \approx 6.2 \times 10^4 \text{ s} \ll 3.6 \times 10^5 \text{ s}, \quad (27)$$

远小于虚拟时间上限，也说明算法不可能因死循环而被强制终止。实际演练中整局虚拟时间约 3300 s，现实程序运行时间 1.7 ~ 2.4 s，相对 1200 s 的限制有三个数量级的余量。

## 6.7 实验结果与消融

图 6 给出一局完整任务的行走轨迹，可直观看到巡站与目标处理如何交错进行。所有本地实验均使用与开发阶段不重叠的新种子做留出验证，并逐例核验：清除数等于真实源数、每次成功清除的实际距离  $\leq 20 \text{ m}$ 、可行多边形与包围圆均包含真源、逐动作时间与 (1) 一致。

表 9 第三问历次结构性改进的配对实验（单位：虚拟秒/源，两版均 100% 清除）

| 改进                | 数据集          | 对照     | 候选     | 下降     |
|-------------------|--------------|--------|--------|--------|
| 单次测向 → 两次测向       | 40 例，498 源   | 977.71 | 764.71 | 21.79% |
| 动态处理顺序 + 共享测站     | 100 例，1310 源 | 723.82 | 653.11 | 9.77%  |
| 25 站网格 → 7 站解析覆盖  | 100 例，1308 源 | 654.53 | 346.62 | 47.04% |
| 环半径 1150 + 混合贪心调度 | 100 例，1315 源 | 343.73 | 263.86 | 23.24% |
| （同上，压力集）          | 70 例，890 源   | 336.31 | 281.83 | 16.20% |
| 圆心补测精化            | 100 例留出      | 262.6  | 260.0  | 1.0%   |

各行使用的案例集合不同，不能把各行降幅连乘得到一个“总提升”。

### 【此处插图】图 5：第三问一局完整轨迹

画法建议：取一局具有代表性的演练（例如  $n = 14$ 、 $\bar{T} = 266.6 \text{ s/源}$ ），画出 1800 m 目标圆、7 个测站、机器狗的完整行走轨迹（按时间用渐变色，起点 (0,0) 标出），每个源的真实位置用叉号、成功清除点用圆圈，失败的光学尝试点用小灰点；右上角小图给出累计虚拟时间随动作序号的阶梯曲线，并按动作类型着色。另可并排放一张对照图：25 站旧版本在同一案例上的轨迹，直观展示路程差异。

图 6 第三问典型局的行走轨迹与时间累积

表 9 中收益最大的一行是测站布局（47.04%），这与第 3.2 节的固定成本分析相吻合。特别地，混合贪心一行的成本分解显示：候选版平均少走 5946 m（约 1189 s），代价



是平均多做 17.3 次检测与 18.3 次失败清除（约 141 s）。这正体现了第四、节以来的一贯取舍——用少量额外测量换取大量移动节省，而不是机械地减少动作次数。

表 10 第三问被否决的改进方案（均完成完整配对实验，清除率均为 100%）

| 方案                                               | 对照 s/源  | 候选 s/源  | 否决原因                                                     |
|--------------------------------------------------|---------|---------|----------------------------------------------------------|
| 两步前瞻调度 $J(a) = C(p, a) + \lambda \min_b C(a, b)$ | 263.920 | 292.795 | 总移动 12583 $\rightarrow$ 14452 m, $P_{95}$ 亦恶化；复杂度上升而性能下降 |
| 非凸可行域（保留 no_signal 挖洞后的非凸单元）                     | 263.920 | 264.943 | 单元碎片化，平均光学格数 100.1 $\rightarrow$ 102.3，区域缩小未转化为更短路径      |
| 自适应第二测点（ $5 \times 4$ 离散集合）                      | 263.920 | 265.518 | 均值与最坏值均恶化，见第五、节                                          |
| 主动搜索（按未知区域覆盖收益选站）                                | 251.70  | 260.95  | 提前减少搜索后已发现目标测向不足，光学成本上升                                  |

## 6.8 理论下界与优化空间

为判断“还能优化多少”，本文建立一个下界模型：任何满足覆盖保证的策略都至少要 (i) 走遍 7 个测站（下界取该站集的最短哈密顿路，目标处理段用随机点巡回的渐近公式  $0.7124\sqrt{NA}$  估计不可避免的额外行程）；(ii) 在所有站上排除空频道（未占用频道数  $\times$  站数  $\times 6$  s，已发现频道平均扫半数站）；(iii) 对每个源至少一次第二测点往返与一次成功清除。结果见表 11。

表 11 第三问的时间下界与现有方案的差距（单位：虚拟秒）

| 源数 $n$ | 路线下界 | 扫描下界 | 每源附加 | 总下界  | 下界 s/源 | 现有本地 s/源 | 差距 |
|--------|------|------|------|------|--------|----------|----|
| 10     | 1874 | 750  | 310  | 2934 | 293    | 310      | 6% |
| 13     | 2030 | 700  | 403  | 3133 | 241    | 265      | 9% |
| 16     | 2180 | 696  | 496  | 3372 | 211    | 226      | 7% |

现有方案距下界只有 6%  $\sim$  9%，剩余差距来自巡站与目标处理交错的调度损失以及最坏情形余量。结合定理 7（站数不能再减）与推论 3（环半径已贴下界），本文认为第三问的优化重点已从“搜索覆盖”转向“定位精度与光学覆盖开销”，继续堆叠更复杂的调度器不再是有效方向。

## 七、 问题四：混合全向源与定向源的检测与清除

### 7.1 第三问的覆盖条件为什么不能照搬

第四问中每个源可能是定向源，其有效覆盖范围是以定向方向  $d$  为轴、张角  $180^\circ$  的闭半平面。于是“可见”的判据从单纯的距离变成距离与方向的合取：

$$\text{测站 } s \text{ 能发现位于 } g \text{、朝向 } d \text{ 的源} \iff \|s - g\| \leq \rho \text{ 且 } d^\top(s - g) \geq 0. \quad (28)$$

这带来两个本质变化：

1. “附近有一个站”不再足够. 离源最近的那个站可能落在天线背面。
2. `no_signal` 不再携带距离信息. 在  $s$  处收不到频道  $c$ ，可能是距离超限，也可能只是背向。因此定理 8 的圆盘排除在第四问不再适用，必须在实现上加以隔离——这是两问算法最重要的一处不对称。

记

$$\mathcal{N}_\rho(g) = \{s \in \mathcal{S} \mid \|s - g\| \leq \rho\}. \quad (29)$$

### 7.2 方向覆盖的充要条件

**定理 9 (凸包等价条件)** 位于  $g$  的源对任意未知朝向  $d$  都存在可见测站，当且仅当

$$g \in \text{conv } \mathcal{N}_\rho(g). \quad (30)$$

**证明.** (充分性) 设  $g \in \text{conv } \mathcal{N}_\rho(g)$ ，则存在  $s_1, \dots, s_k \in \mathcal{N}_\rho(g)$  与  $\lambda_i \geq 0$ ， $\sum \lambda_i = 1$ ，使  $g = \sum \lambda_i s_i$ 。于是对任意单位向量  $d$ ，

$$\sum_i \lambda_i d^\top(s_i - g) = d^\top \left( \sum_i \lambda_i s_i - g \right) = 0.$$

一组和为零的加权项不可能全部严格为负，故存在  $i$  使  $d^\top(s_i - g) \geq 0$ ；又  $\|s_i - g\| \leq \rho$ ，由 (28) 该站可见。

(必要性) 设  $g \notin \text{conv } \mathcal{N}_\rho(g)$ 。 $\text{conv } \mathcal{N}_\rho(g)$  是紧凸集，由凸集分离定理存在单位向量  $d$  与  $\alpha > 0$  使  $d^\top(s - g) \leq -\alpha < 0$  对一切  $s \in \mathcal{N}_\rho(g)$  成立。取该  $d$  为定向方向，则所有距离足够近的站都落在天线背面，而距离超过  $\rho$  的站本来就收不到，故无可见站。□

取  $\rho = \rho_{\min} = 1000 \text{ m}$ ，(30) 对区域内每一点成立即给出保证发现的充要条件。

**与三角剖分条件的比较.** 早期版本 (49 站 600 m 方格、27 站 950 m 三角网格、25 站双环) 使用的是一个充分条件：把区域剖分成直径小于  $\rho_{\min}$  的三角形 (或方格)，源必落在某个单元内，而单元顶点既在接收范围内、又由凸组合关系保证至少一个位于前发射半平面。这一条件要求相邻站之间的距离都小于  $\rho_{\min}$ ，因而站数下不去。

定理 9 放松了这一点： $\mathcal{N}_\rho(g)$  中的站不必两两相邻，只要  $g$  落在它们的凸包内即可。正是这一差别使测站数得以从 25 个减到 22 个。事实上，当前 22 站布局中内环 (990, 0) 与外环 (1616, 933) 的距离达  $1123.6\text{ m} > 1000\text{ m}$ ，任何基于“边长小于 1000 的剖分”的论证都无法覆盖它，而凸包条件依然成立。

### 7.3 连续域的网络证书

(30) 是对连续域中每一点的要求，不能靠有限采样“验证”。本文用一个带余量的网络证书把连续域问题化为有限检查。

**定理 10 (网络证书)** 取网格步长  $h$ ，令  $\delta = h/\sqrt{2}$ 。若对覆盖  $\mathcal{D}$  外扩  $\delta$  的每一个网格点  $g$  都有

$$B(g, \delta) \subseteq \text{conv } \mathcal{N}_{\rho-\delta}(g), \quad (31)$$

则  $\mathcal{D}$  中任意一点  $p$  都满足  $p \in \text{conv } \mathcal{N}_\rho(p)$ 。

**证明.** 任取  $p \in \mathcal{D}$ ，设  $g$  为距  $p$  最近的网格点，则  $\|p - g\| \leq h/\sqrt{2} = \delta$ 。其一，对任意  $s \in \mathcal{N}_{\rho-\delta}(g)$ ，有  $\|s - p\| \leq \|s - g\| + \|g - p\| \leq (\rho - \delta) + \delta = \rho$ ，故  $\mathcal{N}_{\rho-\delta}(g) \subseteq \mathcal{N}_\rho(p)$ ，从而  $\text{conv } \mathcal{N}_{\rho-\delta}(g) \subseteq \text{conv } \mathcal{N}_\rho(p)$ 。其二， $p \in B(g, \delta) \subseteq \text{conv } \mathcal{N}_{\rho-\delta}(g)$ 。二者合并即得  $p \in \text{conv } \mathcal{N}_\rho(p)$ 。□

(31) 中“包含一个半径  $\delta$  的圆盘”这一要求等价于  $g$  到凸包各支撑超平面的距离都不小于  $\delta$ ，即余量  $\text{slack}(g) \geq \delta$ ，可由凸包的超平面表示直接算出（负值表示  $g$  在凸包外）。

**认证结果.** 布局搜索的完整结果见表 13，最终采用的布局见图 7。对该 22 站布局取  $h = 1\text{ m}$ 、 $\delta = 0.7071\text{ m}$ ，检查 10 186 813 个网格点，全部通过，最小余量为  $1.7331\text{ m} > \delta$ （最坏点位于  $(-1738, -471)$ ），计算耗时约 12 s。运行时程序还会用 20 m 粗网格再做一次凸包检查（余量 2.417 m），测站列表不满足覆盖时拒绝运行。

### 7.4 最终布局与站数下界

表 12 第四问采用的 22 站认证布局

| 组成  | 站数 | 半径/m | 角度                                    |
|-----|----|------|---------------------------------------|
| 圆心站 | 1  | 0    | —                                     |
| 内环  | 9  | 990  | $0^\circ, 40^\circ, \dots, 320^\circ$ |
| 外环  | 12 | 1866 | $0^\circ, 30^\circ, \dots, 330^\circ$ |

最近邻巡回长度 17940.2 m（约 3588 s）；相邻内环站间距 677.1 m；相邻外环站间距 966.0 m。

**引理 3 (外环至少 12 站)** 目标圆边界上的点  $g$  要满足 (30), 必须存在位于  $g$  外侧的测站; 所有“外侧站”所成多边形的内切圆半径须不小于  $R = 1800\text{ m}$ 。若该多边形为正  $n$  边形, 则其边长为  $2R \tan(\pi/n)$ 。  $n = 11$  时边长为  $1057.1\text{ m} > \rho_{\min}$ , 相邻两站之间的边界段上存在无法被同时“夹住”的点;  $n = 12$  时边长  $964.6\text{ m} < \rho_{\min}$ 。故外环至少需要 12 个站。

**引理 4 (圆心站必需)** 考虑距圆心约  $300\text{ m}$ 、朝外发射的源。在没有圆心站时,  $\mathcal{N}_{\rho_{\min}}(g)$  只包含近侧的若干内环站, 它们全部位于  $g$  的同一侧,  $g \notin \text{conv } \mathcal{N}_{\rho_{\min}}(g)$ , 由定理 9 存在使其不可见的朝向。

**引理 5 (内环至少 9 站)** 内环取 8 站时相邻站相距  $2 \times 990 \sin 22.5^\circ = 757.7\text{ m}$ , 与外环围成的四边形在  $(-100, 100)$  附近存在这样的点: 只有 3 个站落在其  $1000\text{ m}$  范围内, 且该点位于这三站凸包之外, 网格证书给出的余量为  $-0.5$  到  $-5.9\text{ m}$ 。内环取 9 站时余量转为  $2.42\text{ m}$ ; 继续增大内环半径又会使内外环间距超过  $1000\text{ m}$ 。

表 13 布局搜索结果 (严格证书取  $h = 1\text{ m}$ 、 $\delta = 0.7071\text{ m}$ )

| 布局                             | 站数        | 边界最小余量/m         | 严格证书                 |
|--------------------------------|-----------|------------------|----------------------|
| 圆心 +12@999 +12@1863.5<br>(上一版) | 25        | 0.0 (解析三角剖分证明成立) | 通过                   |
| 圆心 +9@990 +12@1866 (采用)        | <b>22</b> | <b>2.42</b>      | <b>通过, 最小余量 1.73</b> |
| 圆心 +8@990 +12@1866             | 21        | -0.5             | 不通过                  |
| 圆心 +7@990 +12@1866             | 20        | -29              | 不通过                  |
| 三核 +6@1050 +12@1866 等          | 20 ~ 22   | -8 ~ -43         | 不通过                  |
| 随机局部搜索所得布局                     | 19 ~ 22   | -42 ~ -92        | 不通过                  |
| 圆心 +6@1200 +8@2000 等稀疏布局       | 14 ~ 19   | -176 ~ -596      | 不通过                  |

【此处插图】图 6: 第四问 22 站认证布局与方向覆盖余量

画法建议: 左子图画  $1800\text{ m}$  目标圆、22 个测站 (圆心 / 内环 9 个 / 外环 12 个用不同标记), 并画出最近邻巡回路径 ( $17940\text{ m}$ ); 对一个典型边界点  $g$  画出  $\mathcal{N}_{1000}(g)$  及其凸包, 直观展示  $g$  落在凸包内。右子图画方向覆盖余量  $\text{slack}(g)$  在目标圆上的热力图 (顺序色标, 0 处用醒目分界色), 标出最小余量点  $(-1738, -471)$  及其值  $1.73\text{ m}$ ; 可加一个子图对比 21 站时出现的负余量区域。

图 7 22 站布局的方向覆盖证书

## 7.5 在线层：选择性测量、有界等待与认证条带覆盖

第四问的定位层与第三问共用第 6.3 节的可行域递推与目标圆外包，但去掉定理 8 的圆盘排除。在此基础上加入下列针对定向源的改进。

### 7.5.1 选择性重测

到达一个测站后，未发现的频道必须全部检测（这是终止证明的要求，不可省略）；但对已发现的频道，重复测量常常价值很低。设当前站为  $s$ 、目标可行域为  $P_c$ ，只有同时满足

$$\text{dist}(s, P_c) \leq 950 \text{ m} \quad \text{且} \quad \left( \sin \hat{\psi} \geq 0.3 \quad \text{或} \quad \min_{p \in P_c} \|s - p\| \geq 300 \text{ m} \right) \quad (32)$$

时才重测，其中  $\hat{\psi}$  是以  $c_*(P_c)$  为顶点、由已有采样点与  $s$  张成的估计交会角， $P_c$  为已有采样点集。已经可以认证清除或已收到 **near** 的目标直接跳过。

### 7.5.2 有界等待

若某目标只有一条方位线，而后续巡站路线上正好存在能提供第二条方位线的测站（距可行域  $\leq 950 \text{ m}$ ），则可以推迟处理，等待顺手可及的观测，从而避免一次专程往返。为避免无限等待与错误等待，加了三重限制：只等待一次；首次发现点距当前位置超过  $1400 \text{ m}$  时不等待；在目标圆之外首次发现的目标不等待——因为外环站首次发现意味着该源很可能朝外发射，后续站往往全部落在天线背面。

### 7.5.3 圆心补测精化

与第三问同理，但阈值更紧： $R_* \in (20, 150] \text{ m}$  时先走到  $c_*(P_c)$  补测一次，若仍不足则沿新方位垂线横移  $3r_c = 60 \text{ m}$  再补一次。

### 7.5.4 认证条带覆盖

$25 \text{ m}$  方格兜底（(25)）只利用了清除圆的内接正方形，面积利用率为  $2/\pi \approx 63.7\%$ 。本文改用沿首测方向的条带矩形覆盖：把  $P_c$  沿  $\mathbf{u}(\theta)$  切成宽  $w$  的条，再在每条内沿横向切成高

$$h(w) = 2\sqrt{r'^2 - \left(\frac{w}{2}\right)^2}, \quad r' = r_c - 0.1 = 19.9 \text{ m} \quad (33)$$

的矩形，取矩形中心为清除点。此时矩形四角到中心的距离恰为  $\sqrt{(w/2)^2 + (h/2)^2} = r' < r_c$ ，由凸性整个矩形都落在清除圆内，覆盖性得证。 $w$  在  $\{1.25, 1.5, 1.7, 1.9\} \times r'$  中取；例如  $w = 1.5r' = 29.85 \text{ m}$  时  $h = 26.32 \text{ m}$ ，单点覆盖面积  $785.7 \text{ m}^2$ ，比  $25 \text{ m}$  方格的  $625 \text{ m}^2$  高出  $25.7\%$ 。两种覆盖的对比见图 8。

【此处插图】图 7：认证条带覆盖与 25 m 方格覆盖的对比

画法建议：同一个细长可行域  $P_c$  上并排两种覆盖。左图为 25 m 方格：画出与  $P_c$  相交的所有方格、每格中心的清除点与蛇形路径，标注格数与路径长度；右图为条带覆盖：沿首测方向切条（ $w = 1.5r' = 29.85\text{ m}$ ），每条内按  $h = 26.32\text{ m}$  切矩形，画出矩形、中心点与半径 19.9 m 的清除圆（显示四角恰在圆上），标注点数与路径长度。下方用一行文字给出两者按 (34) 算出的成本，说明门控如何二选一。

图 8 两种具有连续覆盖证明的光学清除点列

### 7.5.5 扫描成本门控与 near 即时清除

条带覆盖并非总是更优（条带更宽意味着列数少但每列更长，路径未必更短）。因此算法同时生成方格路线与各个  $w$  对应的条带路线，按

$$\hat{C}_{\text{route}} = \frac{\|x_1 - p_{\text{cur}}\| + \sum_i \|x_{i+1} - x_i\|}{v} + |\{x_i\}| \cdot t_f \quad (34)$$

取最小者。同样的思想用于补测决策：若估计的光学覆盖成本已经不高于“走到补测点再测”的成本，就直接进入光学清除，跳过价值不足的补测。最后，收到 **near** 时在当前测站立即清除，而不是等到本站扫描结束——因为真源就在 5 m 之内，任何推迟都只会增加往返。

## 7.6 实验结果

第四问历次结构性改进的配对实验见表 14，最后一次布局升级（25 → 22 站）的留出验证见表 15，典型局的轨迹与逐例配对散点见图 9。

表 14 第四问历次结构性改进的配对实验（单位：虚拟秒/源，两版均 100% 清除）

| 改进                    | 数据集          | 对照     | 候选     | 下降     |
|-----------------------|--------------|--------|--------|--------|
| 49 站方格 → 27 站三角网格     | 100 例，1310 源 | 928.46 | 767.64 | 17.32% |
| （同上，压力集）              | 50 例，670 源   | 916.49 | 727.66 | 20.60% |
| 27 站 → 25 站双环 + 混合调度  | 100 例随机      | 792.16 | 566.30 | 28.51% |
| 选择性重测 + 有界等待 + 精化     | 220 例，2908 源 | 534.05 | 499.98 | 6.38%  |
| 认证条带 + near 即清 + 成本门控 | 450 例，5935 源 | 499.45 | 493.55 | 1.18%  |

表 15 显示 22 站在全部五类场景上都优于 25 站，而且在最困难的“边界向外”“全部定向”两类压力场景上改善最明显（30.8 与 31.2 s/源）——这与“外环负担了约 60%

表 15 25 站 → 22 站的留出验证（新种子 2026100000 起，与全部开发种子不重叠；220 例全部清除）

| 场景     | 案例数 | 25 站均值 | 22 站均值 | 25 站 P <sub>95</sub> | 22 站 P <sub>95</sub> | 22 站更快 |
|--------|-----|--------|--------|----------------------|----------------------|--------|
| 随机     | 100 | 494.5  | 478.4  | 675.5                | 659.6                | 73/100 |
| 边界向外   | 30  | 588.7  | 557.9  | 725.5                | 683.8                | 24/30  |
| 最小接收半径 | 30  | 517.0  | 502.7  | 677.0                | 611.1                | 20/30  |
| 全部定向   | 30  | 606.1  | 574.9  | 776.5                | 726.3                | 26/30  |
| 圆心聚集   | 30  | 450.9  | 431.1  | 598.2                | 568.8                | 28/30  |

按源数分层（随机集）：10 源 666 → 642，13 源 491 → 475，16 源 361 → 346（虚拟秒/源）。

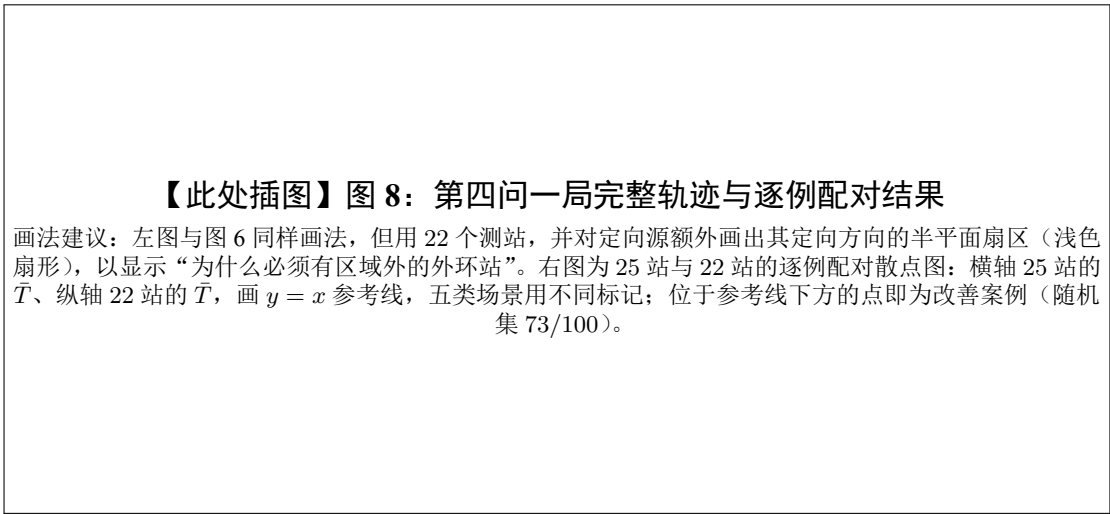


图 9 第四问典型局轨迹与 25 站 / 22 站逐例配对

的搜索到达移动”的诊断一致。

7.7 被否决的改进路线

本文把所有完成了完整配对实验但最终未采用的方案列在表 16。它们构成本文的消融实验，也说明了为什么最终算法是现在这个形态。

7.8 理论下界与“放弃保证”的代价

下界. 以  $n = 13$  为例：巡站  $17940\text{ m} \div 5 = 3588\text{ s}$ ；扫描约 7 个空频道  $\times 22\text{ 站} \times 6\text{ s}$  加上已发现频道在被发现前平均扫半数站，合计约 1800 s；到达与清除每源约 26 s 加 5 s，合计约 400 s。总下界约 5800 s，即约 445 s/ 源。现有方案本地 475、官方演练约 490 ~ 515，距下界约 8% ~ 10%。

若放弃覆盖保证. 本文用 40 万个随机定向源做蒙特卡洛，估计削减测站后的漏检率（表 17）。可见放弃保证确实能换来时间，但代价是整局出现漏源的概率迅速上升：16 站

表 16 第四问被否决的改进方案（全部完成完整实验，清除率均为 100%）

| 方案                                                            | 实验结果与否决原因                                                                                                                                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 绕行代价阈值调度<br>$D_{\text{detour}} = d(x, g) + d(g, s) - d(x, s)$ | 在 27 站与 25 站上各做 1050 次仿真，最佳阈值 A500 分别恶化 8.54% 与 9.15%， $P_{95}$ 同步恶化，89/100 个随机案例退步。“返回主路线”移动确实降了 16%，但光学扫描移动 +49.7%、目标测量移动 +88.0%。 <b>局部绕路指标不能代替整局目标函数。</b> |
| 价值门控的重复测量裁剪（B1）                                               | 测量次数 -14.36%、平均时间 -3.35%，但光学扫描距离 +39%，最坏案例 +296.49 s、额外光学距离 +2786 m。裁剪过度会让少数困难源失去关键方向而进入长距离搜索。                                                               |
| 光学风险保护的测量裁剪（B1.5）                                             | 在 B1 前加 path / attempts 风险门；最佳参数使光学距离增幅压到 2.17%、平均改善 1.72%，但出现新的最坏案例 +950.86 s（某频道只拿到 1 条方位便进入 84 次光学尝试）。 <b>平均值改善不足以抵消尾部风险。</b>                             |
| 固定巡回路线与 2-opt                                                 | 当前“最近未访问站”路线长 18233.1 m；最近邻 +2-opt 与 100 个确定性多起点 +2-opt 给出同样的 18233.1 m。现有路线已与简单确定性方法找到的最好解持平，不替换。                                                           |
| 低价值测站逐一删除                                                     | 在 1120 个案例上统计每站的新源发现、新方向贡献与到达成本；对 25 站逐一做留一实验。虽然若干外环站平均信息价值偏低，但每个外环站都能构造独占方向覆盖的反例， <b>没有任何站获得连续域安全删除证书。</b>                                                  |
| 运行时动态跳站                                                       | 只有在“剩余测站仍能对所有未发现频道提供完整连续覆盖证明”时才允许跳站。1000 个案例中安全证书授权的跳站次数为 0：证书足够安全但过于保守，无实际收益，正式配置关闭。                                                                        |
| 主动搜索（位置-方向联合证书）                                               | 用保守单元维护联合证书，按覆盖收益选站。原型曾存在“使用真实源数决定补测”与“有限探针代替连续证明”两处研究有效性缺陷，修正后仍未胜过正式的混合环调度。                                                                                 |

时每局期望漏检 0.14 个，意味着约七局就有一局清除率低于 100%。按本文的准则（清除率是硬约束），这些方案一律不采用；此处列出是为了量化“保证”的价格。

表 17 放弃方向覆盖保证的收益与风险（40 万随机定向源蒙特卡洛，13 源、半数定向）

| 布局             | 站数 | 单定向源漏检率 | 每局期望漏检 | 估计 s/源 |
|----------------|----|---------|--------|--------|
| 圆心 +9 + 12（采用） | 22 | 0       | 0      | ≈ 490  |
| 圆心 +9 + 8 外环   | 18 | 0.4%    | 0.03   | ≈ 445  |
| 圆心 +9 + 6 外环   | 16 | 2.1%    | 0.14   | ≈ 410  |
| 圆心 +12 内环      | 13 | 17.8%   | 1.16   | ≈ 330  |



## 八、 演练测试与正式测试结果

### 8.1 测试环境与客户端

机器狗程序用 Python 实现，通过 HTTP+JSON 与官方模拟器通信（默认 `http://127.0.0.1:2026`）。客户端遵守附件 2 的全部要求：以参赛队号作为 `robot_id`，每个新动作使用新的 `request_id`，串行发送、逐次等待响应，网络中断后重试时复用原请求内容与原 `request_id`；未知结果的动作标记为 `uncertain` 并禁止发出不同的后续动作，避免重复计时或状态错位。客户端按 `/enter` 返回的 `remaining_real_duration_s` 设置现实截止时间，并在本地同步记录逐动作日志、配置哈希与定位证书，用于事后核对。

### 8.2 演练测试结果

表 18 与表 19 给出在官方模拟器演练模块上的逐局结果。演练结束后模拟器界面显示的干扰源总数与机器狗的成功清除数一致，故

$$\text{被清除干扰源个数的比例} = \frac{N_c}{n} = 100\% \quad (\text{全部局次}). \quad (35)$$

表 18 问题三演练测试逐局结果（官方模拟器）

| 序号      | 时间戳            | 清除个数 | 虚拟总时间/s | 平均定位清除时间/s  | 程序运行时间/s | 访问站数 | 结束方式   |
|---------|----------------|------|---------|-------------|----------|------|--------|
| 1       | 09-11 22:03:30 | 16   | 3202.3  | 200.14      | 2.08     | 6    | 数量上界   |
| 2       | 09-11 22:03:46 | 15   | 3418.0  | 227.86      | 2.42     | 7    | 覆盖完成   |
| 3       | 09-11 22:04:02 | 11   | 3130.5  | 284.60      | 2.20     | 7    | 覆盖完成   |
| 4       | 09-11 22:04:16 | 14   | 3365.4  | 240.38      | 2.28     | 7    | 覆盖完成   |
| 5       | 09-11 22:04:29 | 14   | 3646.4  | 260.46      | 2.22     | 7    | 覆盖完成   |
| 6       | 09-11 22:04:40 | 12   | 3141.7  | 261.81      | 2.31     | 7    | 覆盖完成   |
| 7       | 09-11 22:04:53 | 11   | 2700.6  | 245.51      | 1.70     | 7    | 覆盖完成   |
| 8       | 09-11 22:05:06 | 10   | 2925.2  | 292.52      | 1.66     | 7    | 覆盖完成   |
| 9       | 09-11 22:05:21 | 15   | 3594.1  | 239.60      | 2.44     | 7    | 覆盖完成   |
| 10      | 09-11 22:05:34 | 10   | 2933.6  | 293.36      | 1.86     | 7    | 覆盖完成   |
| 11*     | 09-12 15:01:37 | 15   | 3303.2  | 220.22      | 2.09     | 7    | 覆盖完成   |
| 12*     | 09-12 15:01:54 | 14   | 3732.4  | 266.60      | 2.02     | 7    | 覆盖完成   |
| 13*     | 09-12 15:02:09 | 15   | 3488.9  | 232.59      | 2.05     | 7    | 覆盖完成   |
| 14*     | 09-12 15:02:21 | 13   | 3344.9  | 257.30      | 1.94     | 7    | 覆盖完成   |
| 合计 / 均值 |                | 185  | 3281.2  | 251.6（案例均值） | 2.09     | —    | 全部主动退出 |

\* 为加入圆心补测精化后的最终版本；合并口径（总时间除以总源数）为 248.3 s/ 源。

表 19 问题四演练测试逐局结果（官方模拟器，22 站认证布局）

| 序号      | 时间戳            | 清除个数 | 虚拟总时间/s | 平均定位清除时间/s  | 程序运行时间/s | 访问站数 | 结束方式   |
|---------|----------------|------|---------|-------------|----------|------|--------|
| 1       | 09-12 14:57:20 | 14   | 6334.5  | 452.46      | 3.42     | 22   | 覆盖完成   |
| 2       | 09-12 14:57:35 | 11   | 5986.5  | 544.23      | 3.83     | 22   | 覆盖完成   |
| 3       | 09-12 14:57:49 | 15   | 7347.2  | 489.81      | 4.51     | 22   | 覆盖完成   |
| 4       | 09-12 14:58:05 | 10   | 6539.0  | 653.90      | 3.98     | 22   | 覆盖完成   |
| 5       | 09-12 14:58:52 | 15   | 6223.3  | 414.89      | 3.20     | 22   | 覆盖完成   |
| 6       | 09-12 14:59:06 | 12   | 6234.7  | 519.56      | 2.53     | 22   | 覆盖完成   |
| 7       | 09-12 14:59:19 | 11   | 6282.9  | 571.17      | 3.77     | 22   | 覆盖完成   |
| 8       | 09-12 14:59:32 | 15   | 5933.7  | 395.58      | 3.53     | 22   | 覆盖完成   |
| 9       | 09-12 14:59:47 | 10   | 6214.6  | 621.46      | 3.97     | 22   | 覆盖完成   |
| 10      | 09-12 15:00:16 | 10   | 6197.4  | 619.74      | 3.48     | 22   | 覆盖完成   |
| 合计 / 均值 |                | 123  | 6329.4  | 528.3（案例均值） | 3.62     | —    | 全部主动退出 |

合并口径为 514.6 s/ 源；整局虚拟时间的标准差仅 395 s（相对波动 6.2%）。

### 8.3 本地模型与官方模拟器的一致性

本地合成模拟器是所有配对实验的基础，因此它与官方模拟器是否一致至关重要。把官方演练的 10 局第四问结果按源数分层，与本地在同一源数下的预测比较，结果见表 20：五个分层的最大偏差为 23s/ 源（5.6%），三个分层的偏差在 2s/ 源以内。这说明本地配对实验的结论可以外推到官方环境。

表 20 问题四官方演练与本地预测的分层比较

| 源数 $n$ | 局数 | 官方均值 s/源 | 本地预测 s/源 | 偏差    |
|--------|----|----------|----------|-------|
| 10     | 3  | 631.7    | 642      | -10.3 |
| 11     | 2  | 557.7    | 568      | -10.3 |
| 12     | 1  | 519.6    | 518      | +1.6  |
| 14     | 1  | 452.5    | 452      | +0.5  |
| 15     | 3  | 433.4    | 410      | +23.4 |

图 10 把两问的演练成绩按源数与时间构成做了汇总。

### 8.4 正式测试结果

三次正式测试的结果分别按题目表 1 的格式列于表 21 与表 22。

【待填写】三次正式测试完成后，按模拟器界面的测试案例编码与本地 `summary.json` 中的 `cleared_count`、`average_per_cleared_s`、`program_elapsed_s` 填入下列两表，并删除本提示框。

### 8.5 成绩的正确解读

三次正式测试是三个独立随机案例，样本量极小，因此必须说明成绩的口径：

【此处插图】图 9：演练成绩的结构分析

画法建议：三个子图。(a) 散点图：横轴源数  $n$ 、纵轴整局虚拟总时间，第三问与第四问分别用不同颜色，叠加回归直线  $T_{Q3} = 1886 + 106n$ 、 $T_{Q4} = 5765 + 46n$  及其 95% 带，直观展示“总时间主要由固定成本决定”。(b) 同样的横轴、纵轴改为平均定位清除时间  $\bar{T}$ ，叠加双曲线  $\bar{T} = T_0/n + k$ ，说明分母效应。(c) 堆叠柱状图：第三问与第四问的时间构成（移动 / 检测 / 切频 / 光学失败 / 激光），数据取自表 2。

图 10 演练成绩随源数的变化与时间构成

表 21 问题三正式测试结果

| 测试案例编码    | 清除干扰源个数 | 平均定位清除时间/s | 程序运行时间/s |
|-----------|---------|------------|----------|
| 测试 1 案例编码 |         |            |          |
| 测试 2 案例编码 |         |            |          |
| 测试 3 案例编码 |         |            |          |

表 22 问题四正式测试结果

| 测试案例编码    | 清除干扰源个数 | 平均定位清除时间/s | 程序运行时间/s |
|-----------|---------|------------|----------|
| 测试 1 案例编码 |         |            |          |
| 测试 2 案例编码 |         |            |          |
| 测试 3 案例编码 |         |            |          |

1. 三次成绩的差异主要由抽到的源数决定，而不是算法稳定性。由 (4)，整局虚拟总时间的相对波动只有 6% 左右（第四问  $6329 \pm 395$  s），但  $\bar{T} = T/n$  在  $n = 10$  与  $n = 16$  之间可以相差近一倍。因此建议同时报告整局虚拟总时间与平均定位清除时间两个口径。
2. 平均值的两种口径不能混用。“各局  $T_i/n_i$  再取算术平均”（案例均值）与“ $\sum T_i / \sum n_i$ ”（合并均值）是不同的量，前者每局等权、后者按源数加权。本文所有对比都在同一口径下进行并在表注中标明。
3. 清除比例是第一位的。只有在  $N_c = n$  的前提下比较  $\bar{T}$  才有意义；若某局未清完， $\bar{T}$  反而可能因分母变小而“变好”，必须单列说明而不能混入平均。
4. 程序运行时间不是算法效率指标。演练中程序运行时间为 1.7 ~ 4.5 s，相对 1200 s 的限制有两到三个数量级的余量，主要由 HTTP 往返次数决定，与虚拟时间没有可

比性。

## 九、模型的检验、灵敏度分析与评价

### 9.1 正确性检验的内容

本文所有实验都不是只看平均时间，而是逐例核验下列九项：

- (1) 成功清除数是否等于该案例的真实源数；
- (2) 每次成功清除时，清除点到真源的实际距离是否  $\leq 20\text{ m}$ ；
- (3) 每个时刻的可行多边形  $P_c$  与其最小包围圆是否确实包含真源（即 (23)）；
- (4) 逐条动作重新累加移动、切频、检测、光学与激光时间，与日志时钟的差异是否  $\leq 10^{-6}\text{ s}$ ；
- (5) 测站布局是否在连续域上满足覆盖条件（而不是只在有限采样点上）；
- (6) 边界源、near、 $0^\circ/360^\circ$  跨界、1000 m 接收边界、5 m 近场边界、20 m 背向光学清除、空集 / 无界 / 线段 / 单点等退化几何是否正确处理；
- (7) 不受本轮改动影响的另一问，其完整动作序列是否逐例保持一致（跨问回归）；
- (8) 候选版本是否存在时间退步案例，最大退步是多少；
- (9) 失败运行是否被独立保留并说明，而不是删除。

第 (3) 项尤其重要。第 6.3 节记录的那次失败正是“清除数看似正确、几何证书却已失效”的情形，说明结果正确不能替代不变量检查。

### 9.2 灵敏度分析

#### 9.2.1 对测向误差界的灵敏度

改变实际误差幅度、同时把算法的误差界同步放宽为“幅度  $+0.005^\circ$ ”，每档使用相同的种子，结果见表 23（取自基线 B1 版本，用于观察趋势）。

表 23 测向误差幅度的灵敏度（同一批种子，单位：虚拟秒/源，均 100% 清除）

| 实际误差幅度          | 算法误差界         | 问题三    | 问题四     |
|-----------------|---------------|--------|---------|
| $\pm 0.5^\circ$ | $0.505^\circ$ | 732.38 | 966.58  |
| $\pm 1.0^\circ$ | $1.005^\circ$ | 741.00 | 987.94  |
| $\pm 1.5^\circ$ | $1.505^\circ$ | 745.92 | 1015.95 |

误差幅度加倍只带来 2%（问题三）与 5%（问题四）的时间增长，说明在误差界被正确告知的前提下，方案对误差幅度不敏感。原因是：可行域的尺度与  $\varepsilon$  成正比，但总时间的主体是与  $\varepsilon$  无关的巡站与扫描固定成本。

但误差界失配是危险的。上表是“正确知道更大误差界”的灵敏度，不是“低估误差仍能工作”的证据。本文专门构造了真实偏差达  $1.5^\circ$  而算法只容许  $1.005^\circ$  的算例，结果出现正的半平面违约  $12.958908\text{ m}$ ——真源被排出了可行域，包含性不变量被破坏。这正是算法采用保守界  $\varepsilon = 1.005^\circ$ （而不是恰好  $1^\circ$ ）的原因：吸收“误差先产生、再按两位小数取整”可能带来的额外偏差。一旦正式测试中出现空交集或“认证清除失败”，正确的做法是排查取整、角度约定、坐标与计时，而不是擅自放宽阈值。

### 9.2.2 对最小接收半径 $\rho_{\min}$ 的灵敏度

这是全文最关键的一个敏感参数，因为两问的覆盖保证都以  $\rho_{\min} = 1000\text{ m}$  为前提。

- **问题三.** 由定理 6，七站布局可行当且仅当  $\min_r d_{\max}(r) = R/2 \leq \rho_{\min}$ ，即

$$\rho_{\min} \geq \frac{R}{2} = 900\text{ m}. \quad (36)$$

题目给出的  $1000\text{ m}$  相对该临界值有  $11.1\%$  的余量。当  $\rho_{\min}$  下降时，环半径的可行区间迅速收窄： $\rho_{\min} = 1000$  时为  $[1122.96, 1732.05]$ ， $\rho_{\min} = 950$  时收窄为  $[1254.7, 1645.4]$ ， $\rho_{\min} = 900$  时退化为唯一值  $1558.85\text{ m}$ 。

- **问题四.** 22 站认证布局的余量则薄得多。用  $20\text{ m}$  粗网格计算方向覆盖余量随  $\rho_{\min}$  的变化（表 24），该布局在  $\rho_{\min} \geq 994\text{ m}$  时仍然通过，在  $990\text{ m}$  时失效，即余量约为  $6\text{ m}$ （ $0.6\%$ ）。

表 24 方向覆盖余量对最小接收半径的灵敏度（ $20\text{ m}$  粗网格； $-\infty$  表示存在无法覆盖的点）

| $\rho_{\min}/\text{m}$ | 1000  | 998       | 996       | 994       | 990       | 985       | 980       |
|------------------------|-------|-----------|-----------|-----------|-----------|-----------|-----------|
| 22 站（采用）余量 / m         | 2.417 | 2.417     | 2.417     | 0.482     | $-\infty$ | $-\infty$ | $-\infty$ |
| 25 站（解析）余量 / m         | 0.000 | $-\infty$ | $-\infty$ | $-\infty$ | $-\infty$ | $-\infty$ | $-\infty$ |

一个值得强调的结论是：经过优化搜索得到的 22 站布局，不仅比解析构造的 25 站更快，对  $\rho_{\min}$  的余量也更大（ $6\text{ m}$  对  $< 1\text{ m}$ ）。原因是 25 站方案的内环半径取  $999\text{ m}$ ，圆心站到内环站的距离恰好只比  $\rho_{\min}$  小  $1\text{ m}$ ，三角剖分是“贴着边”成立的；而 22 站方案是在凸包充要条件下做最大化余量的搜索得到的。

尽管如此，两者的余量都不大。因此程序保留了一个显式的回退开关：删除配置中的 `q4_station_list` 即恢复 25 站解析布局；若进一步怀疑  $\rho_{\min}$  的真实取值，应重新运行定理 10 的网格证书并按新的  $\rho_{\min}$  重新搜索布局。

### 9.2.3 对光学格距与其他阈值的灵敏度

光学兜底的正确性要求  $h_{\text{opt}}/\sqrt{2} \leq r_c$ ，即

$$h_{\text{opt}} \leq 20\sqrt{2} = 28.2843 \text{ m}; \quad (37)$$

当前取 25 m，余量 13%。条带覆盖则要求矩形外接圆半径  $\leq r_c$ ，实现中取  $r' = 19.9 \text{ m}$ ，留出 0.1 m 的浮点余量。认证清除阈值取  $20 \text{ m} - 10^{-6}$ 。这些阈值若被误改到超限，覆盖保证立即失效，因此程序在启动时统一校验，不满足则拒绝运行。

### 9.3 稳健性（压力场景）

除随机案例外，本文设计了五类压力场景：源全部贴近区域边界且取最小接收半径、源在圆心聚集、全体取最小接收半径、第四问全部为朝外的定向源、固定  $\pm 1^\circ$  偏差与光滑空间相关偏差。表 15 给出了第四问五类场景的完整结果：220 例全部清除，22 站在每一类上都优于 25 站。第三问的压力集（70 例、890 源）同样全部清除，只有 2 例轻微退步，最大退步 2.72 s/源。

### 9.4 模型的优点

- (1) 把“一定能清完”做成了可证明的性质，而不是经验观察。定理 5 与定理 9 把终止条件建立在连续域的几何覆盖上；定理 10 把连续域要求化为可执行的有限检查；推论 1 与 (25)、(33) 保证了清除动作的确定性成功。整条链路上没有任何一步依赖“试几次没信号就认为没有”。
- (2) 用集员方法正确处理了有界误差。题目只给出误差的界而没有分布，本文相应地只使用凸多边形交与最小包围圆，不引入无法验证的概率假设，且所有近似都取保守外包方向（目标圆外切多边形、圆盘排除取凸包外近似），确保包含性不变量单调成立。
- (3) 优化目标锚定在真实时间上。由第 3.2 节的诊断确定“固定成本主导”，进而把主要力气放在测站布局这一真正的杠杆上，而不是凭直觉调参；对每一项改动都用同一批案例、同一误差场做配对实验，并同时报告均值、 $P_{95}$ 、退步案例数与最大退步。
- (4) 给出了下界，从而知道离极限还有多远。表 11 与第 7.8 节表明现有方案距理论下界约 6% ~ 10%；定理 7 与第 7.4 节说明站数已不能再减。这使“是否还值得继续优化”有了明确答案。
- (5) 完整保留了失败记录。表 10、表 16 共列出 11 条经完整实验后被否决的路线，以及一次几何证书失效及其修复。这些既是消融实验，也是结论可信度的一部分。

## 9.5 模型的缺点与局限

- (1) **调度层是启发式的，没有最优性证明.** 混合贪心只比较一步的移动代价，没有把未来的测量结果、光学路径与切频成本纳入；两步前瞻的尝试反而变慢（表 10），但这只说明该特定前瞻方案并不理想，不说明贪心已最优。
- (2) **第四问的 `no_signal` 信息尚未被利用.** 一次“收不到”其实蕴含“距离超限或背向”的析取信息，理论上可以在“位置  $\times$  朝向”的联合状态空间上做保守排除。本文实现了原型（位置-方向联合证书），但尚未胜过现行方案，因此没有接入。这是最有价值的未完成方向。
- (3) **覆盖余量偏薄.** 如表 24 所示，22 站布局对  $\rho_{\min}$  只有约 6 m 的余量。若官方的接收半径下界并非严格的 1000 m，保证可能失效。
- (4) **本地分布假设未必与官方一致.** 假设 6 中的均匀分布、半数定向等设定是本文自己的假设。虽然表 20 显示本地预测与官方演练吻合良好，但这只是 10 局的证据，不能视为分布一致的证明。
- (5) **结论依赖题设的理想化物理.** 静止源、无遮挡直线行走、确定性阈值、误差有界——任何一条被打破（例如存在反射造成的多径，或源可移动），本文的包含性不变量与覆盖证明都需要重建。

## 9.6 模型的推广

本文的方法框架并不局限于本题的参数，可以直接推广到以下情形：

- **区域形状与规模改变.** 定理 6 与推论 3 给出的是  $R$  与  $\rho_{\min}$  的一般关系；对非圆形区域，只需把定理 10 的网格证书套用到新的区域上重新搜索布局即可，证书本身与区域形状无关。
- **更窄的定向波束.** 若定向源的覆盖角从  $180^\circ$  收窄为  $2\beta < 180^\circ$ ，定理 9 的凸包条件需换成“ $g$  落在  $\mathcal{N}_\rho(g)$  中任意  $\beta$ -锥内至少有一站”的更强条件，站数会显著上升；此时定理 10 的网格证书框架仍然适用，只需更换单点判据。
- **多机器狗协同.** 目标函数 (1) 变为多机的 makespan，覆盖条件不变，只需把测站集划分给若干机器狗并最小化最大巡回长度（即 min-max  $k$ -TSP）。由于固定成本主导，理想情况下  $k$  只狗可把总时间压到约  $1/k$ 。
- **其他“搜索-定位-清除”任务.** 例如水下声源搜寻、林区火点定位、地下管线泄漏排查，都具有“方位有界误差、作用距离有下界、处置有作用半径”的同构结构，本文的四层框架可以整体移植。

## AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色与表述打磨、 $\text{\LaTeX}$  排版与公式规范化、以及程序语法层面的调试；问题的分析、建模、定理证明、算法设计与全部实验均由参赛队员独立完成，详细使用情况见支撑材料。

## 参考文献

- [1] K. Bezdek. Über einige Kreisüberdeckungen [J]. Beiträge zur Algebra und Geometrie, 1983, 14: 7–13.
- [2] K. Bezdek. Über einige optimale Konfigurationen von Kreisen [J]. Ann. Univ. Sci. Budapest. Eötvös Sect. Math., 1984, 27: 143–151.
- [3] H. W. E. Jung. Über die kleinste Kugel, die eine räumliche Figur einschließt [J]. Journal für die reine und angewandte Mathematik, 1901, 123: 241–257.
- [4] E. Welzl. Smallest enclosing disks (balls and ellipsoids) [M]. New Results and New Trends in Computer Science, LNCS 555, Springer, 1991: 359–370.
- [5] J. Beardwood, J. H. Halton, J. M. Hammersley. The shortest path through many points [J]. Mathematical Proceedings of the Cambridge Philosophical Society, 1959, 55(4): 299–327.
- [6] R. T. Rockafellar. Convex Analysis [M]. Princeton University Press, 1970.
- [7] G. Fejes Tóth. Thinnest covering of a circle by eight, nine, or ten congruent circles [J]. Combinatorial and Computational Geometry, MSRI Publications 52, 2005: 361–376.
- [8] D. J. Torrieri. Statistical theory of passive location systems [J]. IEEE Transactions on Aerospace and Electronic Systems, 1984, AES-20(2): 183–198.
- [9] 2026 年高教社杯全国大学生数学建模竞赛 B 题《无线电干扰源的快速自动定位与清除》及附件 1、附件 2.
- [10] L. P. Kaelbling, M. L. Littman, A. R. Cassandra. Planning and acting in partially observable stochastic domains [J]. Artificial Intelligence, 1998, 101(1–2): 99–134.
- [11] M. Milanese, A. Vicino. Optimal estimation theory for dynamic systems with set membership uncertainty: an overview [J]. Automatica, 1991, 27(6): 997–1009.
- [12] R. Kershner. The number of circles covering a set [J]. American Journal of Mathematics, 1939, 61(3): 665–671.



## A 支撑材料清单

表 25 支撑材料目录说明

| 文件 / 目录                                  | 内容                                                             |
|------------------------------------------|----------------------------------------------------------------|
| code/model.py                            | 有界误差几何：楔形半平面、多边形裁剪、半平面交的空 / 无界 / 有界判定、直径、最小包围圆、光学格、测站布局生成与覆盖校验 |
| code/optimized.py                        | 在线策略主体：轨迹维护、共享测站扫描、认证清除与光学兜底入口                                 |
| code/q3_policy.py                        | 第三问专用：目标圆外切约束、no_signal 圆盘排除、第二测点、圆心补测、混合贪心调度                  |
| code/q4_policy.py                        | 第四问专用：22 站列表校验与凸包方向覆盖检查、选择性重测、有界等待、补测精化、条带清除                   |
| code/q4_optical.py                       | 认证条带覆盖与扫描路线成本比较                                                |
| code/<br>practice_robot.py               | 官方模拟器 HTTP 客户端：串行发送、超时原样重试、未决动作保护、逐动作日志与预算控制                   |
| code/config.json                         | 当前全部算法参数与 22 站坐标                                               |
| code/layout_opt.py、<br>code/certify22.py | 方向覆盖余量计算与 $h = 1\text{m}$ 网格证书                                 |
| logs/（三次正式测试）                            | 模拟器导出的加密行为日志，文件名未作任何修改                                         |
| AI 工具使用详情.pdf                            | 按 2026 年 AI 工具使用规定提交的详情说明                                      |

运行方式：安装 Python 3.10 及以上与 `numpy`、`scipy`、`requests`，在模拟器接口就绪后执行

```
python practice_robot.py --question 3 --robot-id < 队号 > --case-code < 案例编码 > --connect
```

（第四问将 `--question` 改为 4）。不加 `--connect` 时程序不会发送任何请求，便于离线检查。

## B 关键参数配置

## C 调试记录：一次边界顶点的浮点失稳

在第三问圆盘排除约束的首轮验证中，种子 2031091108 的案例报出 `Localization certificate excluded true source by 9.7374 m`：该局虽然最终清除了全部 15 个源，但中间的几何证书已经失效。根因是重复施加圆盘排除时，一个理论上位于边界的交点被浮点判为落在圆盘内部，相应二次方程的根又略微超出线段参数范围，导致边界顶点被错误丢弃，多次裁剪后可行域出现不保守的收缩。修复后改用平方距离容差保留边界顶点、对接近端点的根做范围容许与夹取、点数不足或凸包退化时保留输入多边

表 26 算法参数及其来源

| 参数                                | 取值           | 作用                             | 来源      |
|-----------------------------------|--------------|--------------------------------|---------|
| region_radius_m                   | 1800         | 目标区域半径                         | 题目      |
| receiver_radius_min_m             | 1000         | 接收半径下界，覆盖证明的基准                 | 题目      |
| receiver_radius_max_m             | 1500         | 接收半径上界，初始楔形长度                  | 题目      |
| bearing_bound_deg                 | 1.005        | 保守误差界                          | 假设 3    |
| clear_m                           | 20           | 认证清除半径                         | 题目      |
| near_m                            | 5            | 近场阈值                           | 题目      |
| optical_grid_m                    | 25           | 光学方格边长， $25/\sqrt{2} < 20$     | (25)    |
| q3_survey_layout                  | hexagon      | 第三问七站布局                        | 定理 7    |
| q3_ring_radius_m                  | 1150         | 第三问环半径，可行区间 [1122.96, 1732.05] | 推论 3    |
| q3_policy                         | mixed        | 搜索与目标混合贪心调度                    | 第 6.5 节 |
| q3_second_forward/lateral_m       | 200 / 100    | 第二测点偏移                         | 第五、节    |
| q3_refine_max_m                   | 300          | 圆心补测的适用上限                      | 第 6.4 节 |
| q3_refine_offset_m                | 60           | 补测横移距离                         | 第 6.4 节 |
| q4_policy                         | mixed_ring   | 第四问联合调度                        | 第 7.5 节 |
| q4_station_list                   | 22 点         | 认证布局，删除即回退 25 站                | 表 12    |
| q4_selective_remeasure            | true         | 选择性重测                          | (32)    |
| q4_remeasure_min_sine             | 0.3          | 重测的最小交会角正弦                     | (32)    |
| q4_remeasure_min_gap_m            | 300          | 重测的最小测点间距                      | (32)    |
| q4_station_help_m                 | 950          | 判定测站对目标是否有价值                   | (32)    |
| q4_wait_for_survey / q4_wait_mode | true / inner | 有界等待，仅限内环发现的目标                 | 第 7.5 节 |
| q4_wait_stale_m                   | 1400         | 超过该距离不再等待                      | 第 7.5 节 |
| q4_refine_max_m                   | 150          | 第四问补测的适用上限                     | 第 7.5 节 |
| q4_strip_cover                    | true         | 认证条带覆盖                         | (33)    |
| q4_scan_cost_gate                 | true         | 扫描成本门控                         | (34)    |
| q4_immediate_near                 | true         | 收到 near 立即清除                   | 第 7.5 节 |

形，并补充了重复约束的回归测试。本例说明“结果全部清除”不能替代对中间几何不变量的检查（正文第 6.3 节）。

## D 核心源程序

以下为完整的可运行源程序。为便于核对，文件内容与支撑材料中的副本逐字节一致。

### 4.1 有界误差几何与测站布局: model.py

```

1  """Bounded-error geometry and online policy. No hidden-source access here."""
2  import itertools, math
3  import numpy as np
4  from scipy.optimize import linprog
5
6  def direction(deg):
7      a=np.deg2rad(deg);return np.array([np.cos(a),np.sin(a)])
8
9  def bearing_halfplanes(point, deg, error=1.005):
10     lo,hi=direction(deg-error),direction(deg+error)
11     A=np.array([[lo[1],-lo[0]],[-hi[1],hi[0]]])
12     return A,A@np.asarray(point)
13

```

```

14 def clip(poly,A,b,tol=1e-8):
15     poly=np.asarray(poly,dtype=float).reshape(-1,2)
16     for a,c in zip(A,b):
17         if not len(poly):return poly
18         new=[]
19         for p,q in zip(poly,np.roll(poly,-1,axis=0)):
20             fp,fq=a@p-c,a@q-c
21             if fp<=tol:new.append(p)
22             if (fp<=tol)!=(fq<=tol):new.append(p+(q-p)*fp/(fp-fq))
23         poly=np.array(new).reshape(-1,2)
24     return poly
25
26 def polygon_from_halfplanes(A,b):
27     """Classify empty/unbounded/bounded without an artificial bounding box."""
28     A,b=np.asarray(A,float),np.asarray(b,float)
29     feasible=linprog([0,0],A_ub=A,b_ub=b,bounds=[(None,None)]*2,method='highs')
30     if feasible.status==2:return {'status':'empty','vertices':np.empty((0,2)), 'diameter':None}
31     if not feasible.success:raise RuntimeError(feasible.message)
32     for objective in ([1,0],[-1,0],[0,1],[0,-1]):
33         r=linprog(objective,A_ub=A,b_ub=b,bounds=[(None,None)]*2,method='highs')
34         if r.status==3:return {'status':'unbounded','vertices':np.empty((0,2)), 'diameter':math.
35             inf}
36         if not r.success:raise RuntimeError(r.message)
37     vertices=[]
38     for i,j in itertools.combinations(range(len(A)),2):
39         M=A[[i,j]]
40         if abs(np.linalg.det(M))<1e-12:continue
41         x=np.linalg.solve(M,b[[i,j]])
42         if np.all(A@x<=b+1e-7) and not any(np.linalg.norm(x-v)<1e-7 for v in vertices):vertices
43             .append(x)
44     v=np.array(vertices).reshape(-1,2)
45     if not len(v):raise RuntimeError('Numerically degenerate feasible polyhedron')
46     c=v.mean(axis=0);v=v[np.argsort(np.arctan2(v[:,1]-c[1],v[:,0]-c[0]))]
47     return {'status':'bounded','vertices':v,'diameter':diameter(v)}
48
49 def diameter(v):
50     v=np.asarray(v);return float(np.sqrt(np.max(np.sum((v[:,None]-v[None,:])**2,axis=2))))
51
52 def enclosing_circle(v):
53     """Exact candidate enumeration; at most three support vertices in 2D."""
54     v=np.asarray(v);best=(None,math.inf)
55     candidates=list(v)
56     candidates.extend((p+q)/2 for p,q in itertools.combinations(v,2))
57     for p,q,r in itertools.combinations(v,3):
58         M=2*np.array([q-p,r-p])
59         if abs(np.linalg.det(M))>1e-10:
60             candidates.append(np.linalg.solve(M,np.array([q@q-p@p,r@r-p@p])))
61     for c in candidates:

```

```

60     rad=float(np.max(np.linalg.norm(v-c,axis=1)))
61     if rad<best[1]:best=(c,rad)
62     return best
63
64 def initial_polygon(p,deg,cfg):
65     u=direction(deg);v=np.array([-u[1],u[0]])
66     r=cfg['receiver_radius_max_m'];w=r*np.tan(np.deg2rad(cfg['bearing_bound_deg']))
67     return np.array([p,p+r*u-w*v,p+r*u+w*v])
68
69 def optical_cells(poly,origin,deg,h):
70     """Every square intersecting polygon; square circumradius h/sqrt(2)."""
71     u=direction(deg);v=np.array([-u[1],u[0]]);B=np.stack([u,v],axis=1)
72     local=(poly-origin)@B
73     lows=np.floor(local.min(axis=0)/h-.5).astype(int)
74     highs=np.ceil(local.max(axis=0)/h+.5).astype(int)
75     cells=[]
76     A=np.array([[1,0],[-1,0],[0,1],[0,-1]])
77     for i in range(lows[0],highs[0]+1):
78         js=list(range(lows[1],highs[1]+1))
79         if i%2:js.reverse()
80         for j in js:
81             c=np.array([i*h,j*h]);b=np.array([c[0]+h/2,-c[0]+h/2,c[1]+h/2,-c[1]+h/2])
82             if len(clip(local,A,b)):cells.append(origin+B@c)
83     return cells
84
85 def triangular_survey_points(cfg):
86     """Vertices of every closed triangular cell intersecting the target disk.
87
88     The centroid of one cell is at the origin. See docs/experiment2_report.md.
89     Keep exterior vertices: clipping stations to the disk breaks direction coverage.
90     """
91     radius=cfg['region_radius_m']
92     spacing=cfg.get('q4_triangle_spacing_m',950)
93     reception=cfg['receiver_radius_min_m']
94     if not all(math.isfinite(v) and v>0 for v in (radius,spacing,reception)):
95         raise ValueError('Triangle survey requires positive finite distances')
96     if spacing>=reception:
97         raise ValueError('Triangle spacing must be below minimum reception radius')
98     height=spacing*math.sqrt(3)/2
99     offset=np.array([spacing/2,height/3])
100     def vertex(i,j):
101         return np.array([spacing*(i+j/2),height*j])-offset
102     # Includes a full cell beyond each disk bounding-box edge.
103     rows=math.ceil((radius+height/3)/height)+1
104     columns=math.ceil(radius/spacing+rows/2+0.5)+1
105     tolerance=1e-9*max(1.,radius,spacing)
106     selected=set()
107     for j in range(-rows,rows+1):

```

```

108     for i in range(-columns,columns+1):
109         for ids in (((i,j),(i+1,j),(i,j+1)),
110                     ((i+1,j),(i+1,j+1),(i,j+1))):
111             a=np.array([vertex(*ij) for ij in ids])
112             edge=np.roll(a,-1,axis=0)-a
113             cross=edge[:,0]*(-a[:,1])-edge[:,1]*(-a[:,0])
114             inside=bool(np.all(cross>=0))
115             fraction=np.clip(-np.sum(a*edge,axis=1)/np.sum(edge*edge,axis=1),0,1)
116             closest=a+fraction[:,None]*edge
117             if inside or np.any(np.sum(closest*closest,axis=1)<=(radius+tolerance)**2):
118                 selected.update(ids)
119     return sorted((vertex(*ij) for ij in selected),key=lambda p:(p[0],p[1]))
120
121 def worst_uncovered_distance(stations,region,spacing=5.):
122     """Largest distance from any disk point to its nearest station (grid plus rim)."""
123     stations=np.asarray(stations,float)
124     xs=np.arange(-region,region+spacing,spacing);X,Y=np.meshgrid(xs,xs)
125     points=np.column_stack((X.ravel(),Y.ravel()));points=points[np.hypot(points[:,0],points
126    [:,1])<=region]
127     angles=np.linspace(0,2*np.pi,int(2*np.pi*region/spacing)+1,endpoint=False)
128     points=np.vstack((points,region*np.column_stack((np.cos(angles),np.sin(angles)))))
129     worst=0.
130     for start in range(0,len(points),100000):
131         block=points[start:start+100000]
132         worst=max(worst,float(np.min(np.linalg.norm(block[:,None,:]-stations[None,:, :],axis=2),
133         axis=1).max()))
134     return worst+spacing/math.sqrt(2)
135
136 def survey_points(question,cfg):
137     if question==3:
138         layout=cfg.get('q3_survey_layout','grid')
139         if layout=='hexagon':
140             # Center plus six ring stations; validate the continuous bound below.
141             # See docs/experiment3_report.md for the configurable-ring proof.
142             radius=cfg['region_radius_m']
143             if not math.isfinite(radius) or radius<=0 or radius/2>cfg['receiver_radius_min_m']:
144                 raise ValueError('Hexagon survey requires 0 < region radius <= 2 * minimum
145             reception radius')
146             ring=cfg.get('q3_ring_radius_m',radius*math.cos(math.pi/6))
147             if not math.isfinite(ring) or not 0<ring<=math.sqrt(3)*radius:
148                 raise ValueError('Q3 ring radius must be positive, finite, and within the
149             coverage geometry')
150             # Extrema: Voronoi junction and the target-circle sector bisector.
151             worst=max(ring/math.sqrt(3),math.sqrt(radius**2+ring**2-math.sqrt(3)*radius*ring))
152             if worst>cfg['receiver_radius_min_m']-1e-6:
153                 raise ValueError('Q3 ring does not guarantee minimum-radius coverage')
154             return [np.zeros(2)]+[ring*direction(60*k) for k in range(6)]
155         if layout=='six':

```

```

152     # EXPERIMENTAL, NOT GUARANTEED: six unit disks cannot cover a disk of
153     # radius ratio 1.8 (Bezdek's bound is 1.7988), so a thin boundary band
154     # stays farther than the minimum reception radius. The caller must
155     # accept a nonzero miss probability; see docs/q3_six_station_trial.md.
156     stations=np.asarray(cfg.get('q3_station_list'),float)
157     if stations.ndim!=2 or stations.shape[1]!=2 or not np.all(np.isfinite(stations)) or
len(stations)<3:
158         raise ValueError('Q3 six-station layout requires q3_station_list with finite
planar points')
159         allowed=cfg.get('q3_uncovered_tolerance_m')
160         if not (isinstance(allowed,(int,float)) and math.isfinite(allowed) and 0<allowed
<=100):
161             raise ValueError('Q3 six-station layout requires an explicit
q3_uncovered_tolerance_m in (0,100]')
162             worst=worst_uncovered_distance(stations,cfg['region_radius_m'])
163             if worst>cfg['receiver_radius_min_m']+allowed:
164                 raise ValueError(f'Q3 station list leaves points {worst:.1f} m from every
station, beyond the declared tolerance')
165             return list(stations)
166         if layout!='grid':raise ValueError(f'Unknown Q3 survey layout: {layout}')
167 if question==4:
168     layout=cfg.get('q4_survey_layout','grid')
169     if layout=='triangular':return triangular_survey_points(cfg)
170     if layout!='grid':raise ValueError(f'Unknown Q4 survey layout: {layout}')
171     h=cfg[f'q{question}_grid_m'];n=math.ceil(cfg['region_radius_m']/h)
172     return [np.array([i*h,j*h],float) for i in range(-n,n+1) for j in range(-n,n+1)]
173
174 class Policy:
175     def __init__(self,transport,cfg,question,strategy='triangulate'):
176         self.io=transport;self.cfg=cfg;self.question=question;self.strategy=strategy
177         self.pos=np.zeros(2);self.channel=1;self.cleared=set();self.observations={};self.
certificates=[]
178     def measure(self,p,ch):
179         r=self.io.measure(p,ch);self.pos=np.array(p);self.channel=ch;return r
180     def clear(self,p,ch):
181         r=self.io.clear(p,ch);self.pos=np.array(p)
182         if r['clear_result']=='success':self.cleared.add(ch);return True
183         return False
184     def locate(self,ch,p,r):
185         if r['measure_result']=='near':
186             if not self.clear(p,ch):raise RuntimeError('near contradicts clear')
187             return
188         deg=r['svd_deg'];poly=initial_polygon(p,deg,self.cfg)
189         if self.strategy=='triangulate':
190             u=direction(deg);v=np.array([-u[1],u[0]])
191             q=p+self.cfg['second_forward_m']*u+self.cfg['second_lateral_m']*v
192             r2=self.measure(q,ch)
193             if r2['measure_result']=='near':

```

```

194         if not self.clear(q,ch):raise RuntimeError('near contradicts clear')
195         return
196         if r2['measure_result']=='direction':
197             A,b=bearing_halfplanes(q,r2['svd_deg'],self.cfg['bearing_bound_deg']);poly=clip
(poly,A,b)
198             if not len(poly):raise RuntimeError('Inconsistent bearing bounds; stop rather
than fabricate estimate')
199             center,radius=enclosing_circle(poly)
200             self.certificates.append({'channel':ch,'center':center.tolist(),'radius_m':radius,'
diameter_m':diameter(poly),'vertices':poly.tolist()})
201             if radius<=self.cfg['clear_m']-1e-6:
202                 if not self.clear(center,ch):raise RuntimeError('Certified disk clear failed')
203                 return
204             cells=optical_cells(poly,p,deg,self.cfg['optical_grid_m'])
205             if cells and np.linalg.norm(cells[-1]-self.pos)<np.linalg.norm(cells[0]-self.pos):cells
.reverse()
206             for cell in cells:
207                 if self.clear(cell,ch):return
208             raise RuntimeError('Optical cover exhausted without success')
209     def run(self):
210         points=survey_points(self.question,self.cfg);visited=0
211         while points:
212             k=min(range(len(points)),key=lambda i:float(np.sum((points[i]-self.pos)**2)))
213             p=points.pop(k);visited+=1
214             channels=[c for c in range(1,21) if c not in self.cleared]
215             channels.sort(key=lambda c:(c!=self.channel,c))
216             hits=[]
217             for ch in channels:
218                 r=self.measure(p,ch)
219                 if r['measure_result']!='no_signal':hits.append((ch,p.copy(),r))
220             for ch,s,r in hits:
221                 if ch not in self.cleared:self.locate(ch,s,r)
222                 if len(self.cleared)==16:break
223             return {'cleared_count':len(self.cleared),'visited_survey_points':visited,'termination'
:'count_upper_bound' if len(self.cleared)==16 else 'full_coverage'}

```

## 4.2 在线策略主体: optimized.py

```

1  """Route ordering and shared stations; only observed responses enter decisions."""
2  import numpy as np
3  from model import Policy,direction,initial_polygon,clip,bearing_halfplanes,enclosing_circle,
diameter,optical_cells,survey_points
4
5  class OptimizedPolicy(Policy):
6      def __init__(self,transport,cfg,question,variant='shared'):
7          super().__init__(transport,cfg,question,'triangulate')

```

```

8         self.variant=variant;self.tracks={};self.shared_attempts=0;self.shared_hits=0
9         self.q3_planner=None
10        if question==3 and variant=='shared' and cfg.get('q3_policy','legacy')=='mixed':
11            from q3_policy import Q3Planner
12            self.q3_planner=Q3Planner(self)
13        self.planner=self.q3_planner
14        if question==4 and variant=='shared' and cfg.get('q4_policy','legacy')=='mixed_ring':
15            from q4_policy import Q4Planner
16            self.planner=Q4Planner(self)
17    def measure(self,p,ch):
18        response=super().measure(p,ch)
19        if self.planner is not None:self.planner.observe(p,ch,response)
20        return response
21    def make_track(self,ch,p,r):
22        deg=r.get('svd_deg',0.)
23        self.tracks[ch]={ 'origin':p.copy(), 'deg':deg, 'poly':initial_polygon(p,deg,self.cfg), '
count':1, 'near':p.copy() if r['measure_result']=='near' else None, 'extra':0, 'sample_points'
: [p.copy()] }
24        if self.planner is not None:self.planner.constrain(ch)
25    def update(self,ch,p,r):
26        t=self.tracks[ch];t['sample_points'].append(p.copy())
27        if r['measure_result']=='near':t['near']=p.copy();return
28        if r['measure_result']=='direction':
29            A,b=bearing_halfplanes(p,r['svd_deg'],self.cfg['bearing_bound_deg']);t['poly']=clip
(t['poly'],A,b)
30            if not len(t['poly']):raise RuntimeError('Inconsistent observed bearings')
31            t['count']+=1
32        if self.planner is not None:self.planner.constrain(ch)
33    def destination(self,ch):
34        if self.planner is not None:return self.planner.destination(ch)
35        t=self.tracks[ch]
36        if t['near'] is not None:return t['near'],False
37        c,rad=enclosing_circle(t['poly'])
38        if self.variant=='shared' and (rad<=20-1e-6 or t['count']>=2):return c,False
39        u=direction(t['deg']);v=np.array([-u[1],u[0]])
40        candidates=[t['origin']+650*u+sign*250*v for sign in [-1,1]]
41        q=min(candidates,key=lambda x:np.linalg.norm(x-self.pos))
42        return q,True
43    def shared_scan(self,exclude=None):
44        if self.variant!='shared':return
45        station=self.pos.copy()
46        for ch in sorted(self.tracks):
47            if ch in self.cleared or ch==exclude:continue
48            t=self.tracks[ch]
49            if t['near'] is not None or t['count']>=3 or t['extra']>=4:continue
50            if min(np.linalg.norm(station-p) for p in t['sample_points'])<100:continue
51            c,rad=enclosing_circle(t['poly'])
52            if rad<=20 or np.linalg.norm(c-station)>1300:continue

```



```

53     a=c-t['origin'];b=c-station
54     sine=abs(a[0]*b[1]-a[1]*b[0])/max(np.linalg.norm(a)*np.linalg.norm(b),1e-9)
55     if sine<.12:continue
56     t['extra']+=1;self.shared_attempts+=1
57     r=self.measure(station,ch);self.update(ch,station,r)
58     if r['measure_result']!='no_signal':self.shared_hits+=1
59     if r['measure_result']=='near':
60         if not self.clear(station,ch):raise RuntimeError('near clear failed')
61 def finish(self,ch):
62     t=self.tracks[ch]
63     if t['near'] is not None:
64         if not self.clear(t['near'],ch):raise RuntimeError('near clear failed')
65         return
66     poly=t['poly'];c,rad=enclosing_circle(poly)
67     self.certificates.append(dict(channel=ch,center=c.tolist(),radius_m=rad,diameter_m=
diameter(poly),vertices=poly.tolist()))
68     if rad<=20-1e-6:
69         if not self.clear(c,ch):raise RuntimeError('Certified clear failed')
70         return
71     cells=optical_cells(poly,t['origin'],t['deg'],self.cfg['optical_grid_m'])
72     if np.linalg.norm(cells[-1]-self.pos)<np.linalg.norm(cells[0]-self.pos):cells.reverse()
73     for p in cells:
74         if self.clear(p,ch):return
75     raise RuntimeError('Optical cover exhausted')
76 def run(self):
77     if self.planner is not None:return self.planner.run()
78     points=survey_points(self.question,self.cfg);visited=0
79     while points:
80         idx=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-self.pos));p=
points.pop(idx);visited+=1
81         self.tracks={}
82         channels=sorted((c for c in range(1,21) if c not in self.cleared),key=lambda c:(c!=
self.channel,c))
83         for ch in channels:
84             r=self.measure(p,ch)
85             if r['measure_result']!='no_signal':self.make_track(ch,p,r)
86         while any(ch not in self.cleared for ch in self.tracks):
87             remaining=[ch for ch in self.tracks if ch not in self.cleared]
88             ch=min(remaining,key=lambda ch:np.linalg.norm(self.destination(ch)[0]-self.pos)
)
89             q,needs_measure=self.destination(ch)
90             if needs_measure:
91                 r=self.measure(q,ch);self.update(ch,q,r)
92                 self.shared_scan(exclude=ch)
93                 self.finish(ch)
94                 self.shared_scan()
95             if len(self.cleared)==16:break
96     return dict(cleared_count=len(self.cleared),visited_survey_points=visited,termination='

```

```
count_upper_bound' if len(self.cleared)==16 else 'full_coverage',shared_attempts=self.
shared_attempts,shared_hits=self.shared_hits)
```

### 4.3 第三问专用策略: q3\_policy.py

```
1 """Q3-only mixed routing and conservative constraints from observed responses."""
2 import math
3 import numpy as np
4 from scipy.spatial import ConvexHull,QhullError
5 from model import clip,direction,enclosing_circle,survey_points
6
7 def outside_disk_hull(poly,center,radius):
8     """Outer approximation of a convex polygon minus an open disk.
9
10     Extremes of its convex hull are polygon vertices outside the disk or
11     edge/circle intersections. Degenerate numerical cases retain the input.
12     """
13     points=[]
14     squared_tolerance=1e-7*max(1.,radius*radius)
15     for a,b in zip(poly,np.roll(poly,-1,axis=0)):
16         delta=a-center;edge=b-a
17         if delta@delta>=radius*radius-squared_tolerance:points.append(a)
18         aa=edge@edge;bb=2*delta@edge;cc=delta@delta-radius*radius
19         discriminant=bb*bb-4*aa*cc
20         if aa>1e-16 and discriminant>=0:
21             for fraction in ((-bb-math.sqrt(discriminant))/(2*aa),
22                             (-bb+math.sqrt(discriminant))/(2*aa)):
23                 if -1e-9<=fraction<=1+1e-9:points.append(a+np.clip(fraction,0.,1.)*edge)
24     if len(points)<3:return poly
25     points=np.unique(np.asarray(points),axis=0)
26     if len(points)<3:return poly
27     try:return points[ConvexHull(points).vertices]
28     except QhullError:return poly
29
30 class Q3Planner:
31     def __init__(self,policy):
32         self.p=policy
33         self.silent={ch:[] for ch in range(1,21)}
34         angles=np.arange(48)*2*np.pi/48
35         self.region_normals=np.column_stack([np.cos(angles),np.sin(angles)])
36         self.forward=policy.cfg.get('q3_second_forward_m',200.)
37         self.lateral=policy.cfg.get('q3_second_lateral_m',100.)
38         # Round 6: measure at the region center (then a lateral offset) before
39         # falling back to the optical grid, for regions up to this radius.
40         self.refine_max=policy.cfg.get('q3_refine_max_m',300.)
41         self.refine_offset=policy.cfg.get('q3_refine_offset_m',60.)
```

```

42     if not all(math.isfinite(x) and x>0 for x in (self.forward,self.lateral,self.
refine_offset)):
43         raise ValueError('Q3 second-point offsets must be positive and finite')
44     if not (math.isfinite(self.refine_max) and self.refine_max>=0):
45         raise ValueError('Q3 refinement radius must be non-negative and finite')
46 def observe(self,position,ch,response):
47     if response['measure_result']=='no_signal':
48         self.silent[ch].append(np.asarray(position,float).copy())
49 def constrain(self,ch):
50     p=self.p;t=p.tracks[ch]
51     if t['near'] is not None:return
52     # Supporting halfplanes circumscribe the disk, never inscribe it.
53     poly=clip(t['poly'],self.region_normals,
54              np.full(48,p.cfg['region_radius_m']+1e-7))
55     if not len(poly):raise RuntimeError('Bearings contradict the target region')
56     for station in self.silent[ch]:
57         # In Q3 all transmitters are omnidirectional and radius >= R_min.
58         # A slightly smaller excluded disk leaves numerical safety margin.
59         poly=outside_disk_hull(poly,station,p.cfg['receiver_radius_min_m']-1e-4)
60     t['poly']=poly
61 def destination(self,ch):
62     p=self.p;t=p.tracks[ch]
63     if t['near'] is not None:return t['near'],False
64     center,radius=enclosing_circle(t['poly'])
65     if radius<=20-1e-6 or t['count']>=2:return center,False
66     u=direction(t['deg']);v=np.array([-u[1],u[0]])
67     candidates=[t['origin']+self.forward*u+sign*self.lateral*v for sign in (-1,1)]
68     point=min(candidates,key=lambda q:np.linalg.norm(q-p.pos)+np.linalg.norm(q-center))
69     return point,True
70 def refine(self,ch):
71     """Shrink a 20-300 m region with one or two close bearings; certified clears stay exact
. """
72     p=self.p;t=p.tracks[ch]
73     if t['near'] is not None or self.refine_max<=0:return
74     center,radius=enclosing_circle(t['poly'])
75     if not (p.cfg['clear_m']<radius<=self.refine_max):return
76     response=p.measure(center,ch);p.update(ch,center,response)
77     if response['measure_result']!='direction' or t['near'] is not None:return
78     center2,radius2=enclosing_circle(t['poly'])
79     if radius2<=p.cfg['clear_m']-1e-6:return
80     u=direction(response['svd_deg']);v=np.array([-u[1],u[0]])
81     point=min((center+self.refine_offset*v,center-self.refine_offset*v),
82              key=lambda q:np.linalg.norm(q-center2))
83     response=p.measure(point,ch);p.update(ch,point,response)
84 def cost(self,ch):
85     point,needs_measure=self.destination(ch)
86     center,_=enclosing_circle(self.p.tracks[ch]['poly'])
87     return np.linalg.norm(point-self.p.pos)+(np.linalg.norm(point-center) if needs_measure

```

```

else 0.)
88 def run(self):
89     p=self.p;points=survey_points(3,p.cfg);visited=0
90     while points or any(ch not in p.cleared for ch in p.tracks):
91         pending=[ch for ch in p.tracks if ch not in p.cleared]
92         ch=min(pending,key=self.cost) if pending else None
93         index=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-p.pos)) if
points else None
94         if index is not None and (ch is None or np.linalg.norm(points[index]-p.pos)<self.
cost(ch)):
95             station=points.pop(index);visited+=1
96             channels=sorted((c for c in range(1,21) if c not in p.cleared),key=lambda c:(c
!=p.channel,c))
97             for c in channels:
98                 response=p.measure(station,c)
99                 if c in p.tracks:p.update(c,station,response)
100                 elif response['measure_result']!='no_signal':p.make_track(c,station,
response)
101             else:
102                 point,needs_measure=self.destination(ch)
103                 if needs_measure:
104                     response=p.measure(point,ch);p.update(ch,point,response)
105                     p.shared_scan(exclude=ch)
106                     # Refine, then the original certified clear and complete optical fallback.
107                     self.refine(ch)
108                     p.finish(ch);p.shared_scan()
109                 if len(p.cleared)==16:break
110             return dict(cleared_count=len(p.cleared),visited_survey_points=visited,
111                         termination='count_upper_bound' if len(p.cleared)==16 else 'full_coverage',
112                         shared_attempts=p.shared_attempts,shared_hits=p.shared_hits)

```

## 4.4 第四问专用策略：q4\_policy.py

```

1  """Q4 selective measurements and bounded refinement with certified coverage."""
2  import math
3  import numpy as np
4  from model import clip, direction, enclosing_circle, diameter
5  from q4_optical import clearing_route, route_seconds
6
7
8  def ring_stations(cfg):
9      """Triangulate an enclosing dodecagon with edges below reception range.
10
11      Every source is a convex combination of three stations within reception
12      range. Any closed halfplane through that source contains a station.
13      This proves coverage for every unknown antenna orientation.

```

```

14     """
15     radius=cfg['region_radius_m'];reception=cfg['receiver_radius_min_m']
16     inner=999.*radius/1800.
17     outer=radius/math.cos(math.pi/12)
18     bound=max(inner,2*radius*math.tan(math.pi/12),
19               math.sqrt(inner**2+outer**2-2*inner*radius))
20     if not all(math.isfinite(v) and v>0 for v in (radius,reception)) or bound>=reception:
21         raise ValueError('Q4 ring stations do not satisfy the reception certificate')
22     angles=np.arange(12)*2*np.pi/12
23     inner_points=inner*np.column_stack((np.cos(angles),np.sin(angles)))
24     angles=angles+math.pi/12
25     outer_points=outer*np.column_stack((np.cos(angles),np.sin(angles)))
26     return list(np.vstack([[0.,0.],inner_points,outer_points]))
27
28
29 def hull_coverage_slack(stations,cfg,spacing=20.,margin=0.):
30     """Minimum over a disk grid of the exact directional-coverage slack.
31
32     A source at g with any orientation is detected from some station iff g lies
33     in the convex hull of the stations within reception range of g. The slack is
34     the distance from g to that hull boundary (negative outside), capped by the
35     range margin; a rigorous continuum certificate additionally needs the slack
36     to exceed spacing/sqrt(2) with margin=spacing/sqrt(2) (see
37     experiments/q4_layout_search). This coarse check only guards against typos.
38     """
39     from scipy.spatial import ConvexHull
40     S=np.asarray(stations,float);R=cfg['receiver_radius_min_m'];region=cfg['region_radius_m']
41     xs=np.arange(-region,region+spacing,spacing);X,Y=np.meshgrid(xs,xs)
42     G=np.column_stack((X.ravel(),Y.ravel()));G=G[np.hypot(G[:,0],G[:,1])<=region]
43     angles=np.linspace(0,2*np.pi,int(2*np.pi*region/spacing)+1,endpoint=False)
44     G=np.vstack((G,region*np.column_stack((np.cos(angles),np.sin(angles)))))
45     D=np.linalg.norm(G[:,None,:]-S[None,:,:],axis=2);mask=D<=R-margin
46     worst=math.inf;groups={}
47     for i,row in enumerate(map(bytes,np.packbits(mask,axis=1))):groups.setdefault(row, []).
48         append(i)
49     for idx in groups.values():
50         T=S[mask[idx[0]]]
51         if len(T)<3:return -math.inf
52         eq=ConvexHull(T).equations;g=G[idx]
53         worst=min(worst,float((-g@eq[:,2]).T+eq[:,2]).max(axis=1)).min())
54     return worst
55
56 def survey_stations(cfg):
57     """Survey layout: the analytic 25-station rings or an explicit certified list."""
58     explicit=cfg.get('q4_station_list')
59     if explicit is None:return ring_stations(cfg)
60     S=np.asarray(explicit,float)

```

```

61     if S.ndim!=2 or S.shape[1]!=2 or not np.all(np.isfinite(S)) or len(S)<3:
62         raise ValueError('q4_station_list must be a finite list of planar points')
63     if hull_coverage_slack(S,cfg)<0:
64         raise ValueError('q4_station_list fails the directional coverage check')
65     return list(S)
66
67
68 def polygon_distance(poly,point):
69     """Euclidean distance to a nonempty convex polygon, including degeneracies."""
70     poly=np.asarray(poly,float).reshape(-1,2);point=np.asarray(point,float)
71     if not len(poly):raise ValueError('Empty target region')
72     edges=np.roll(poly,-1,axis=0)-poly;delta=point-poly
73     cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
74     area2=np.sum(poly[:,0]*np.roll(poly[:,1],-1)-poly[:,1]*np.roll(poly[:,0],-1))
75     if abs(area2)>1e-12 and (np.all(cross>=0) or np.all(cross<=0)):return 0.
76     length2=np.sum(edges*edges,axis=1)
77     fraction=np.divide(np.sum(delta*edges,axis=1),length2,out=np.zeros(len(poly)),where=length2
78 >0)
79     closest=poly+np.clip(fraction,0,1)[:,:None]*edges
80     return float(np.linalg.norm(closest-point,axis=1).min())
81
82 class Q4Planner:
83     def __init__(self,policy):
84         self.p=policy
85         angles=np.arange(48)*2*np.pi/48
86         self.normals=np.column_stack([np.cos(angles),np.sin(angles)])
87         self.forward=policy.cfg.get('q4_second_forward_m',200.)
88         self.lateral=policy.cfg.get('q4_second_lateral_m',100.)
89         if not all(math.isfinite(v) and v>0 for v in (self.forward,self.lateral)):
90             raise ValueError('Q4 second-point offsets must be positive and finite')
91         cfg=policy.cfg
92         self.clear_m=cfg['clear_m']
93         self.selective=cfg.get('q4_selective_remeasure',True)
94         self.wait=cfg.get('q4_wait_for_survey',True)
95         self.shared=cfg.get('q4_shared_scan',True)
96         self.help_m=cfg.get('q4_station_help_m',950.)
97         self.help_distance=cfg.get('q4_help_distance','polygon')
98         self.min_sine=cfg.get('q4_remeasure_min_sine',0.3)
99         self.min_gap=cfg.get('q4_remeasure_min_gap_m',300.)
100        self.stale_m=cfg.get('q4_wait_stale_m',1400.)
101        self.refine_max=cfg.get('q4_refine_max_m',150.)
102        self.wait_mode=cfg.get('q4_wait_mode','inner')
103        self.strip_cover=cfg.get('q4_strip_cover',False)
104        self.immediate_near=cfg.get('q4_immediate_near',False)
105        self.scan_cost_gate=cfg.get('q4_scan_cost_gate',False)
106        if not all(type(value) is bool for value in (self.selective,self.wait,self.shared)):
107            raise ValueError('Q4 feature switches must be booleans')

```

```

108     if not all(math.isfinite(v) and v>0 for v in (self.clear_m,self.help_m,self.min_gap,
self.stale_m)):
109         raise ValueError('Q4 distances must be positive and finite')
110     if not math.isfinite(self.refine_max) or self.refine_max<0:
111         raise ValueError('Q4 refinement radius must be non-negative and finite')
112     if not math.isfinite(self.min_sine) or not 0<=self.min_sine<=1:
113         raise ValueError('Q4 intersection sine must be between zero and one')
114     if self.help_m>=cfg['receiver_radius_min_m'] or self.help_distance not in ('polygon','
center'):
115         raise ValueError('Invalid Q4 station help criterion')
116     if self.wait_mode not in ('stale','inner','same_side','all_sides'):
117         raise ValueError('Invalid Q4 waiting rule')
118     if not all(type(value) is bool for value in (self.strip_cover,self.immediate_near,self.
scan_cost_gate)):
119         raise ValueError('Q4 optical feature switches must be booleans')
120
121     def observe(self,position,ch,response):
122         # Silence is compatible with a nearby directional transmitter.
123         pass
124
125     def constrain(self,ch):
126         t=self.p.tracks[ch]
127         if t['near'] is not None:return
128         poly=clip(t['poly'],self.normals,
129                 np.full(48,self.p.cfg['region_radius_m']+1e-7))
130         if not len(poly):raise RuntimeError('Bearings contradict the target region')
131         t['poly']=poly
132
133     def destination(self,ch):
134         p=self.p;t=p.tracks[ch]
135         if t['near'] is not None:return t['near'],False
136         center,radius=enclosing_circle(t['poly'])
137         if radius<=self.clear_m-1e-6 or t['count']>=2:return center,False
138         u=direction(t['deg']);v=np.array([-u[1],u[0]])
139         candidates=[t['origin']+self.forward*u+sign*self.lateral*v for sign in (-1,1)]
140         return min(candidates,key=lambda q:np.linalg.norm(q-p.pos)+np.linalg.norm(q-center)),
True
141
142     def cost(self,ch):
143         point,measure=self.destination(ch)
144         center,_=enclosing_circle(self.p.tracks[ch]['poly'])
145         return np.linalg.norm(point-self.p.pos)+(np.linalg.norm(point-center) if measure else
0.)
146
147     def close_enough(self,station,ch):
148         t=self.p.tracks[ch]
149         if self.help_distance=='center':
150             return np.linalg.norm(enclosing_circle(t['poly'])[0]-station)<=self.help_m

```

```

151     return polygon_distance(t['poly'],station)<=self.help_m
152
153 def useful(self,station,ch):
154     if not self.selective:return True
155     t=self.p.tracks[ch]
156     if t['near'] is not None:return False
157     center,radius=enclosing_circle(t['poly'])
158     if radius<=self.clear_m-1e-6 or not self.close_enough(station,ch):return False
159     for previous in t['sample_points']:
160         a=center-previous;b=center-station
161         sine=abs(a[0]*b[1]-a[1]*b[0])/max(np.linalg.norm(a)*np.linalg.norm(b),1e-9)
162         if sine<self.min_sine and np.linalg.norm(station-previous)<self.min_gap:return
False
163     return True
164
165 def ready(self,ch):
166     t=self.p.tracks[ch]
167     if t['near'] is not None:return True
168     radius=enclosing_circle(t['poly'])[1]
169     return radius<=max(self.clear_m-1e-6,self.refine_max)
170
171 def target_cost(self,ch,stations):
172     t=self.p.tracks[ch]
173     if self.ready(ch):
174         center=t['near'] if t['near'] is not None else enclosing_circle(t['poly'])[0]
175         return np.linalg.norm(center-self.p.pos)
176     # Waiting is bounded by the remaining finite survey and distance from first detection.
177     if self.wait and np.linalg.norm(t['origin']-self.p.pos)<self.stale_m:
178         if self.wait_mode=='inner' and np.linalg.norm(t['origin'])>=self.p.cfg['
region_radius_m']:
179             return self.cost(ch)
180         for station in stations:
181             if not self.close_enough(station,ch):continue
182             if self.wait_mode in ('same_side','all_sides'):
183                 probes=t['poly'] if self.wait_mode=='all_sides' else enclosing_circle(t['
poly'])[0][None,:]
184                 a=t['origin']-probes;b=station-probes
185                 cosine=np.sum(a*b,axis=1)/np.maximum(np.linalg.norm(a,axis=1)*np.linalg.
norm(b,axis=1),1e-9)
186                 if np.any(cosine<0.3):continue
187                 return math.inf
188             return self.cost(ch)
189
190 def refine(self,ch):
191     t=self.p.tracks[ch];p=self.p
192     if t['near'] is not None:return
193     center,radius=enclosing_circle(t['poly'])
194     if not self.clear_m<radius<=self.refine_max:return

```



```

195     if self.scan_cost_gate:
196         points,_=clearing_route(t['poly'],t['origin'],t['deg'],p.pos,p.cfg)
197         budget=route_seconds(points,p.pos,p.cfg)
198         travel=np.linalg.norm(center-p.pos)/p.cfg['speed_m_s']
199         if budget<=travel+p.cfg['measure_s']+p.cfg['clear_failure_s']+10:return
200     response=p.measure(center,ch);p.update(ch,center,response)
201     if response['measure_result']!='direction':return
202     new_center,new_radius=enclosing_circle(t['poly'])
203     if new_radius<=self.clear_m-1e-6:return
204     u=direction(response['svd_deg']);side=np.array([-u[1],u[0]])
205     offset=3*self.clear_m
206     candidates=(center+offset*side,center-offset*side)
207     point=min(candidates,key=lambda q:np.linalg.norm(q-new_center))
208     response=p.measure(point,ch);p.update(ch,point,response)
209
210 def finish(self,ch):
211     p=self.p;t=p.tracks[ch]
212     if ch in p.cleared:return
213     if t['near'] is not None:return p.finish(ch)
214     center,radius=enclosing_circle(t['poly'])
215     if radius<=self.clear_m-1e-6 or not self.strip_cover:return p.finish(ch)
216     points,_=clearing_route(t['poly'],t['origin'],t['deg'],p.pos,p.cfg)
217     p.certificates.append(dict(channel=ch,center=center.tolist(),radius_m=radius,
218                               diameter_m=diameter(t['poly']),vertices=t['poly'].tolist()))
219     for point in points:
220         if p.clear(point,ch):return
221     raise RuntimeError('Certified optical strip cover exhausted')
222
223 def run(self):
224     p=self.p;points=survey_stations(p.cfg);visited=0
225     while points or any(ch not in p.cleared for ch in p.tracks):
226         pending=[ch for ch in p.tracks if ch not in p.cleared]
227         costs={ch:self.target_cost(ch,points) for ch in pending}
228         ch=min(pending,key=costs.get) if pending else None
229         index=min(range(len(points)),key=lambda i:np.linalg.norm(points[i]-p.pos)) if
points else None
230         if index is not None and (ch is None or np.linalg.norm(points[index]-p.pos)<costs[
ch]):
231             station=points.pop(index);visited+=1
232             channels=sorted((c for c in range(1,21) if c not in p.cleared and
233                             (c not in p.tracks or self.useful(station,c))),key=lambda c:(c
!=p.channel,c))
234             for c in channels:
235                 response=p.measure(station,c)
236                 if c in p.tracks:p.update(c,station,response)
237                 elif response['measure_result']!='no_signal':p.make_track(c,station,
response)
238                 if self.immediate_near and response['measure_result']=='near':

```

```

239         if not p.clear(station,c):raise RuntimeError('near clear failed')
240         if len(p.cleared)==16:break
241     else:
242         if not self.ready(ch):
243             point,measure=self.destination(ch)
244             if measure:
245                 response=p.measure(point,ch);p.update(ch,point,response)
246                 if self.shared:p.shared_scan(exclude=ch)
247             self.refine(ch)
248             self.finish(ch)
249             if self.shared:p.shared_scan()
250         if len(p.cleared)==16:break
251     return dict(cleared_count=len(p.cleared),visited_survey_points=visited,
252               termination='count_upper_bound' if len(p.cleared)==16 else 'full_coverage',
253               shared_attempts=p.shared_attempts,shared_hits=p.shared_hits)

```

## 4.5 认证条带覆盖: q4\_optical.py

```

1  """Continuous polygon covers for Q4 optical clearing, independent of RF visibility."""
2  import math
3  import numpy as np
4  from model import clip, direction, optical_cells
5
6
7  def route_seconds(points,start,cfg):
8      points=np.asarray(points,float)
9      length=float(np.linalg.norm(points[0]-start)+np.linalg.norm(np.diff(points,axis=0),axis=1).
10                  sum())
11      return length/cfg['speed_m_s']+len(points)*cfg['clear_failure_s']
12
13  def strip_cover(poly,origin,deg,width,radius):
14      """Each clipped slab lies in rectangles whose corners are inside one clear disk."""
15      u=direction(deg);basis=np.stack([u,[-u[1],u[0]]],axis=1)
16      local=(np.asarray(poly)-origin)@basis
17      lo,hi=local[:,0].min(),local[:,0].max()
18      count=max(1,math.ceil((hi-lo)/width))
19      cuts=np.linspace(lo,hi,count+1);columns=[];certificates=[]
20      for left,right in zip(cuts[:-1],cuts[1:]):
21          section=clip(local,[[1.,0.],[-1.,0.]],right,-left)
22          if not len(section):continue
23          bottom,top=section[:,1].min(),section[:,1].max()
24          half_x=(right-left)/2
25          max_height=2*math.sqrt(max(0.,radius*radius-half_x*half_x))
26          if max_height<=0:raise ValueError('Strip wider than clear diameter')
27          rows=max(1,math.ceil((top-bottom)/max_height))

```

```

28     ycuts=np.linspace(bottom,top,rows+1)
29     centers=[]
30     for lower,upper in zip(ycuts[:-1],ycuts[1:]):
31         center=np.array([(left+right)/2,(lower+upper)/2])
32         corners=np.array([[left,lower],[right,lower],[right,upper],[left,upper]])
33         assert np.max(np.linalg.norm(corners-center,axis=1))<=radius+1e-7
34         centers.append(origin+basis@center)
35         certificates.append(dict(center=(origin+basis@center).tolist(),
36                                 corners=(origin+corners@basis.T).tolist()))
37     columns.append(centers)
38     return columns,certificates
39
40
41 def order_columns(columns,start):
42     path=[];position=np.asarray(start)
43     for column in columns:
44         if np.linalg.norm(column[-1]-position)<np.linalg.norm(column[0]-position):column=column
45         [::-1]
46         path.extend(column);position=column[-1]
47     return path
48
49 def clearing_route(poly,origin,deg,start,cfg):
50     old=optical_cells(poly,origin,deg,cfg['optical_grid_m'])
51     if np.linalg.norm(old[-1]-start)<np.linalg.norm(old[0]-start):old.reverse()
52     best=old;cost=route_seconds(old,start,cfg);certificate=None
53     radius=cfg['clear_m']-0.1
54     if radius<=0:return best,certificate
55     for fraction in (1.25,1.5,1.7,1.9):
56         columns,rectangles=strip_cover(poly,origin,deg,fraction*radius,radius)
57         for ordered in (columns,columns[::-1]):
58             candidate=order_columns(ordered,start)
59             candidate_cost=route_seconds(candidate,start,cfg)
60             if candidate_cost<cost-1e-7:
61                 best,cost,certificate=candidate,candidate_cost,rectangles
62     return best,certificate

```

## 4.6 方向覆盖余量与网格证书: layout\_opt.py、certify22.py

```

1  """Search Q4 survey layouts with a margin-certified directional coverage condition.
2
3  Certificate: for every point g of the 1800 m disk there is a set T of stations with
4  |s-g| <= R - m for all s in T and g inside conv(T) with distance >= m to its boundary.
5  Then any closed halfplane through any g' within m of g contains a station within R of g',
6  so every antenna orientation of a source at g' is seen from some station.
7  Grid spacing h certifies the continuum when the slack exceeds h/sqrt(2).

```

```

8 """
9 import sys,math,json,time
10 import numpy as np
11 from scipy.spatial import ConvexHull
12
13 R=1000.;REGION=1800.
14
15 def grid(h):
16     xs=np.arange(-REGION,REGION+h,h);X,Y=np.meshgrid(xs,xs);G=np.column_stack((X.ravel(),Y.
17         ravel()))
18     G=G[np.hypot(G[:,0],G[:,1])<=REGION]
19     ring=np.linspace(0,2*np.pi,int(2*np.pi*REGION/h)+1,endpoint=False)
20     return np.vstack((G,REGION*np.column_stack((np.cos(ring),np.sin(ring)))))
21
22 def slack(S,G,m):
23     """Min over grid of hull slack using stations within R-m; -inf if fewer than 3."""
24     S=np.asarray(S);D=np.linalg.norm(G[:,None,:]-S[None,:,:],axis=2);mask=D<=R-m
25     packed=np.packbits(mask,axis=1);keys={}
26     for i,row in enumerate(map(bytes,packed)):keys.setdefault(row,[]).append(i)
27     third=np.sort(D,axis=1)[:,2] if S.shape[0]>=3 else np.full(len(G),np.inf)
28     out=(R-m)-third # negative when fewer than three stations are in range
29     for row,idx in keys.items():
30         T=S[mask[idx[0]]]
31         if len(T)<3:continue
32         try:eq=ConvexHull(T).equations
33         except Exception:out[idx]=np.minimum(out[idx],-1.);continue
34         g=G[idx];out[idx]=np.minimum(out[idx],-(g@eq[:,2].T+eq[:,2]).max(axis=1))
35     return out
36
37 def objective(S,G,m):
38     s=slack(S,G,m);k=int(np.argmin(s));return float(s[k]),G[k]
39
40 def tour(S):
41     S=[np.zeros(2)]+[np.asarray(p) for p in S];n=len(S);order=[0];left=set(range(1,n))
42     while left:
43         k=min(left,key=lambda i:np.linalg.norm(S[i]-S[order[-1]]));order.append(k);left.remove(
44             k)
45     L=lambda o:sum(np.linalg.norm(S[o[i]]-S[o[i+1]])) for i in range(len(o)-1)
46     improved=True
47     while improved:
48         improved=False
49         for i in range(1,n-1):
50             for j in range(i+1,n):
51                 new=order[:i]+order[i:j+1][::-1]+order[j+1:]
52                 if L(new)<L(order)-1e-9:order=new;improved=True
53     return L(order)
54
55 def ring(n,r,phase=0.):

```

```

54     a=np.arange(n)*2*np.pi/n+phase;return list(np.column_stack((r*np.cos(a),r*np.sin(a))))
55
56 def seeds(N,rng):
57     out=[]
58     for n_out in (9,10,11,12):
59         for center in (0,1):
60             n_in=N-n_out-center
61             if n_in<3:continue
62             for r_in in (700,850,950):
63                 for r_out in (1850,1900,1950):
64                     out.append([np.zeros(2)] if center else [])+ring(n_in,r_in,rng.uniform
65                               (0,2*np.pi))+ring(n_out,r_out,rng.uniform(0,2*np.pi))
66
67     return out
68
69 def optimize(N,budget_s,seed=0,m=20.,h=40.):
70     rng=np.random.default_rng(seed);G=grid(h);best=None;t0=time.time()
71     cands=seeds(N,rng)
72     scored=sorted(((objective(S,G,m)[0],i) for i,S in enumerate(cands)),reverse=True)[:6]
73     pool=[np.array(cands[i]) for _,i in scored]
74     while time.time()-t0<budget_s:
75         for S in pool:
76             cur,_=objective(S,G,m)
77             for _ in range(40):
78                 step=rng.choice([15.,40.,100.]);T=S.copy()
79                 k=rng.integers(len(S)) if rng.random()<0.7 else slice(None)
80                 T[k]+=rng.normal(0,step,size=np.shape(T[k]))
81                 val,_=objective(T,G,m)
82                 if val>cur:S[:]=T;cur=val
83             if best is None or cur>best[0]:best=(cur,S.copy())
84             if time.time()-t0>budget_s:break
85     return best
86
87 if __name__=='__main__':
88     N=int(sys.argv[1]);budget=float(sys.argv[2]);seed=int(sys.argv[3]) if len(sys.argv)>3 else
89     0
90     val,S=optimize(N,budget,seed)
91     fine,worst=objective(S,grid(6.),20.)
92     print(json.dumps(dict(N=N,coarse_slack=val,fine_slack=fine,worst=worst.tolist(),tour_m=tour
93                          (list(S)),stations=S.tolist()))))

```

```

1  """Rigorous grid certificate for the 22-station layout (see docs/q4_layout_search.md)."""
2  import sys,json,math,time;sys.path.insert(0,'experiments/q4_layout_search')
3  import numpy as np
4  from layout_opt import slack
5  S=np.array(json.load(open('experiments/q4_layout_search/ring22_stations.json')));REGION=1800.
6  h=float(sys.argv[1]) if len(sys.argv)>1 else 1.;delta=h/math.sqrt(2);t=time.time()
7  xs=np.arange(-REGION-2*h,REGION+3*h,h);worst=math.inf;wpt=None;count=0

```

```

8 for x0 in xs[::200]:
9     X,Y=np.meshgrid(xs[(xs>=x0)&(xs<x0+200*h)],xs);G=np.column_stack((X.ravel(),Y.ravel()));G=G
    [np.hypot(G[:,0],G[:,1])<=REGION+delta]
10     if not len(G):continue
11     count+=len(G)
12     for i in range(0,len(G),80000):
13         s=slack(S,G[i:i+80000],delta);k=int(np.argmin(s))
14         if s[k]<worst:worst=float(s[k]);wpt=G[i+k]
15 ok=worst>=delta+1e-6
16 res=dict(stations=S.tolist(),grid_h=h,delta=delta,grid_points=count,min_slack=worst,worst_point
    =wpt.tolist(),certified=bool(ok),seconds=time.time()-t)
17 print(json.dumps({k:v for k,v in res.items() if k!='stations'}))
18 json.dump(res,open('experiments/q4_layout_search/ring22_certificate.json','w'),indent=1)

```

## 4.7 模拟器通信客户端: practice\_robot.py

```

1 """Official-protocol practice runner for questions 3 and 4.
2
3 Select the practice module in the simulator yourself. The protocol does not
4 provide a way to detect whether the current session is practice or formal.
5 """
6 import argparse
7 import hashlib
8 import http.client
9 import json
10 import math
11 import socket
12 import sys
13 import time
14 import unicodedata
15 import uuid
16 from datetime import datetime
17 from pathlib import Path
18 from urllib.error import HTTPError, URLError
19 from urllib.parse import urlsplit
20 from urllib.request import ProxyHandler, Request, build_opener
21
22 from model import Policy
23 from optimized import OptimizedPolicy
24
25 class BudgetStop(RuntimeError):
26     pass
27
28 class ProtocolFailure(RuntimeError):
29     pass
30

```

```

31 class UncertainAction(RuntimeError):
32     """A request may already have executed. Do not send a different next action."""
33     pass
34
35 class PracticeHTTP:
36     def __init__(self, robot_id, base_url, log_path, timeout=5., attempts=3, progress=True):
37         self.robot_id=robot_id;self.base_url=base_url.rstrip('/')
38         self.timeout=timeout;self.attempts=attempts;self.progress=progress
39         self.log=Path(log_path).open('x',encoding='utf-8',buffering=1)
40         self.opener=build_opener(ProxyHandler({}))
41         self.session=uuid.uuid4().hex;self.counter=0;self.virtual_time=0.
42         self.deadline=None;self.enter_started=None;self.started=False;self.ended=False
43         self.uncertain=False;self.position=[0.,0.];self.channel=1;self.cleared=set()
44         self.max_virtual=360000.;self.parts=dict(move=0.,switch=0.,measure=0.,optical=0.,laser
=0.)
45         self.accepted_actions=0;self.attempted_requests=0;self.latest_remaining=None
46
47     def record(self,event,**data):
48         self.log.write(json.dumps(dict(event=event,local_time=datetime.now().isoformat(),**data
),ensure_ascii=False,allow_nan=False)+'\n')
49         self.log.flush()
50
51     @staticmethod
52     def number(value,name):
53         if isinstance(value,bool) or not isinstance(value,(int,float)) or not math.isfinite(
value):
54             raise ProtocolFailure(f'Invalid {name}: {value!r}')
55         return float(value)
56
57     def post(self,path,**fields):
58         if self.uncertain:raise UncertainAction('Previous action unresolved; stop this run.')
59         self.counter+=1
60         payload=dict(arena_id='default',robot_id=self.robot_id,request_id=f'{self.session}-{
self.counter}',**fields)
61         raw=json.dumps(payload,ensure_ascii=False,allow_nan=False,separators=(',',':')).encode(
'utf-8')
62         if len(raw)>65536:raise ProtocolFailure('Request too large')
63         # Log before sending so an interruption does not hide a possibly executed action.
64         self.record('request',path=path,payload=payload)
65         last=None
66         for attempt in range(1,self.attempts+1):
67             remaining=self.timeout if self.deadline is None else self.deadline-time.monotonic()
68             reserve=0. if path=='/exit' else 8.
69             if self.deadline is not None and remaining<=reserve:
70                 if attempt>1:
71                     self.uncertain=True;raise UncertainAction('Deadline reached before retry
could resolve action')
72                     raise BudgetStop('Actual remaining runtime is nearly exhausted')

```

```

73     request=Request(self.base_url+path,data=raw,headers={'Content-Type':'application/
json'},method='POST')
74     self.attempted_requests+=1
75     self.uncertain=True
76     try:
77         with self.opener.open(request,timeout=min(self.timeout,max(.05,remaining-
reserve)) if self.deadline else self.timeout) as response:
78             status=response.status;content=response.read()
79         except HTTPError as error:
80             content=error.read().decode('utf-8',errors='replace')
81             self.record('http_error',path=path,request_id=payload['request_id'],status=
error.code,body=content)
82             # Never start a different action after an ambiguous server error.
83             raise ProtocolFailure(f'HTTP {error.code}: {content[:400]}') from error
84         except (URLError,OSError,socket.timeout,http.client.HTTPException) as error:
85             last=error;self.record('transport_error',path=path,request_id=payload['
request_id'],attempt=attempt,error=str(error))
86             if attempt<self.attempts:
87                 time.sleep(.15)
88                 continue
89             raise UncertainAction(f'Connection failed after {attempt} attempts; action
status unknown: {last}') from error
90         try:
91             body=json.loads(content.decode('utf-8'))
92             if not isinstance(body,dict):raise ValueError('Response must be an object')
93         except (ValueError,UnicodeError) as error:
94             self.record('invalid_response',path=path,body=content[:2000].decode('utf-8',
errors='replace'))
95             raise ProtocolFailure('Response is not valid JSON; action status unknown') from
error
96         self.record('response',path=path,request_id=payload['request_id'],status=status,
body=body)
97         if status!=200 or body.get('accepted') is not True:
98             raise ProtocolFailure(f'Request rejected; stop and check simulator: HTTP {
status}, {body}')
99         vt=self.number(body.get('virtual_time_s'),'virtual_time_s')
100         if vt<0 or vt+1e-6<self.virtual_time:raise ProtocolFailure('Virtual clock went
backwards')
101         self.virtual_time=vt;self.accepted_actions+=1
102         # Caller validates the action-specific fields before releasing the guard.
103         return body
104         raise UncertainAction(str(last))
105
106     def enter(self):
107         self.enter_started=time.monotonic();r=self.post('/enter')
108         remaining=self.number(r.get('remaining_real_duration_s'),'remaining_real_duration_s')
109         limit=self.number(r.get('max_virtual_duration_s'),'max_virtual_duration_s')
110         if not 0<=remaining<=1200 or limit<=0:raise ProtocolFailure('Invalid session limits')

```



```

111     self.deadline=self.enter_started+remaining;self.latest_remaining=remaining;self.
max_virtual=limit
112     self.started=True;self.uncertain=False
113     if self.progress:print(f'已进入测试, 实际可用现实时间 {remaining:.0f} 秒.',flush=True)
114     return r
115
116 def check_action(self,p,ch,worst_cost):
117     if not self.started or self.ended:raise ProtocolFailure('Session is not active')
118     if isinstance(ch,bool) or int(ch)!=ch or not 1<=ch<=20:raise ProtocolFailure('Channel
must be an integer in 1..20')
119     p=[self.number(float(v),'coordinate') for v in p]
120     if len(p)!=2 or any(abs(v)>2000000 for v in p):raise ProtocolFailure('Invalid position'
)
121     move=math.dist(p,self.position)/5
122     if self.virtual_time+move+worst_cost>=self.max_virtual-1:raise BudgetStop('Virtual time
budget nearly exhausted')
123     return p,move
124
125 def account(self,before,delta):
126     # Official clock rounds at microsecond resolution. Allow accumulated tiny rounding.
127     if abs(self.virtual_time-before-delta)>0.002:raise ProtocolFailure('Official clock
disagrees with action accounting')
128     self.uncertain=False
129     if self.progress and self.accepted_actions%50==0:
130         print(f'动作 {self.accepted_actions} | 已清除 {len(self.cleared)} | 虚拟时间 {self.
virtual_time:.1f} 秒',flush=True)
131
132 def measure(self,p,ch):
133     p,move=self.check_action(p,ch,6);before=self.virtual_time;switch=int(self.channel!=ch)
134     r=self.post('/measure',position=dict(x=p[0],y=p[1]),channel=int(ch))
135     kind=r.get('measure_result')
136     if kind not in ('direction','near','no_signal'):raise ProtocolFailure('Unknown
measure_result')
137     if kind=='direction':
138         angle=self.number(r.get('svd_deg'),'svd_deg')
139         if not 0<=angle<360:raise ProtocolFailure('Bearing outside [0,360)')
140     self.account(before,move+switch+5);self.position=p;self.channel=int(ch)
141     self.parts['move']+move;self.parts['switch']+switch;self.parts['measure']+5
142     return r
143
144 def clear(self,p,ch):
145     p,move=self.check_action(p,ch,5);before=self.virtual_time
146     r=self.post('/clear',position=dict(x=p[0],y=p[1]),channel=int(ch))
147     kind=r.get('clear_result')
148     if kind not in ('success','no_target_in_range'):raise ProtocolFailure('Unknown
clear_result')
149     success=kind=='success'
150     if success and ch in self.cleared:raise ProtocolFailure('Duplicate successful clear')

```

```

151     self.account(before,move+3+2*success);self.position=p
152     self.parts['move']+move;self.parts['optical']+=3;self.parts['laser']+=2*success
153     if success:
154         self.cleared.add(int(ch))
155         if self.progress:print(f'清除频道 {ch}, 累计 {len(self.cleared)} 个, 虚拟时间 {self.
virtual_time:.1f} 秒。',flush=True)
156     return r
157
158 def exit(self):
159     r=self.post('/exit')
160     if r.get('exit_reason')!='user_exit':raise ProtocolFailure('Unexpected exit reason')
161     self.uncertain=False;self.ended=True;return r
162
163 def arguments(argv=None):
164     parser=argparse.ArgumentParser(description='第3/4问演练程序: 覆盖搜索、路线排序与多目标共用测
站。')
165     parser.add_argument('--question',type=int,choices=[3,4],required=True)
166     parser.add_argument('--robot-id',required=True,help='必须与模拟器登录队号一致')
167     parser.add_argument('--base-url',default='http://127.0.0.1:2026')
168     parser.add_argument('--strategy',choices=['shared','route','triangulate','sweep'],default='
shared')
169     parser.add_argument('--output-dir',default=str(Path(__file__).resolve().parent/'
practice_logs'))
170     parser.add_argument('--timeout',type=float,default=5.)
171     parser.add_argument('--attempts',type=int,default=3)
172     parser.add_argument('--case-code',default='',help='可选: 从模拟器界面抄录, 用于关联日志')
173     parser.add_argument('--connect',action='store_true',help='连接已经启动且接口就绪的演练测试')
174     parser.add_argument('--config',default='config.json',help='配置文件名 (位于程序目录); 实验入
口使用 config_q3_six.json')
175     args=parser.parse_args(argv)
176     if not args.connect:parser.error('未发送任何请求。请先在模拟器启动对应演练, 接口就绪后加 --
connect。')
177     if not args.robot_id or len(args.robot_id.encode('utf-8'))>64 or any(unicodedata.category(
ch).startswith('C') for ch in args.robot_id):parser.error('队号为空、过长或包含不可见字符')
178     url=urlsplit(args.base_url)
179     if url.scheme!='http' or url.hostname not in ('127.0.0.1','localhost','::1') or url.path
not in ('','/') or url.query or url.fragment or url.username or url.password:parser.error('
接口地址必须是本机回环HTTP地址, 例如 http://127.0.0.1:2026')
180     if not math.isfinite(args.timeout) or not .1<=args.timeout<=30 or not 1<=args.attempts<=5:
parser.error('timeout必须在0.1至30秒之间, attempts必须在1至5之间')
181     return args
182
183 def main(argv=None):
184     args=arguments(argv);base=Path(__file__).resolve().parent
185     if Path(args.config).name!=args.config or not (base/args.config).is_file():raise SystemExit
(f'配置文件必须位于程序目录: {args.config}')
186     cfg=json.loads((base/args.config).read_text(encoding='utf-8'))
187     out=Path(args.output_dir);out.mkdir(parents=True,exist_ok=True)

```

```

188 run_dir=out/f'q{args.question}_{datetime.now():%Y%m%d_%H%M%S}_{uuid.uuid4().hex[:8]}'
189 run_dir.mkdir();io=PracticeHTTP(args.robot_id,args.base_url,run_dir/'actions.jsonl',args.
    timeout,args.attempts)
190 meta=dict(question=args.question,strategy=args.strategy,declared_mode='practice',
    mode_verified_by_protocol=False,case_code=args.case_code,config=cfg,python=sys.version,
    code_sha256={name:hashlib.sha256((base/name).read_bytes()).hexdigest() for name in ['
    practice_robot.py','model.py','optimized.py','q3_policy.py','q4_policy.py','q4_optical.py',
    args.config]}},config_file=args.config)
191 (run_dir/'run_config.json').write_text(json.dumps(meta,ensure_ascii=False,indent=2),
    encoding='utf-8')
192 print(f'第{args.question}问, 策略 {args.strategy}, 日志: {run_dir}',flush=True)
193 if args.config!='config.json':print(f'警告: 使用实验配置 {args.config}, 不保证发现全部干扰
    源, 请勿用于正式测试。',flush=True)
194 print('接口无法识别演练/正式模式; 当前应为模拟器中已启动的对应演练。',flush=True)
195 policy=None;result={};status='failed';error=None;code=1
196 try:
197     io.enter()
198     policy=OptimizedPolicy(io,cfg,args.question,args.strategy) if args.strategy in ('shared
    ','route') else Policy(io,cfg,args.question,args.strategy)
199     result=policy.run();io.exit();status='completed';code=0
200 except (Exception,KeyboardInterrupt) as exc:
201     error=f'{type(exc).__name__}: {exc}'
202     io.record('run_error',error=error)
203     # Exit only when the last action is resolved and a live session is known.
204     if io.started and not io.ended and not io.uncertain and io.deadline and time.monotonic
    ()<io.deadline-1:
205         try:io.exit()
206         except Exception as exit_error:io.record('exit_error',error=str(exit_error))
207         if isinstance(exc,KeyboardInterrupt):status='interrupted';code=130
208         elif isinstance(exc,BudgetStop):status='budget_stop';code=2
209         print(f'本次未确认完成: {error}',file=sys.stderr,flush=True)
210 finally:
211     summary=dict(status=status,error=error,**result,cleared_channels=sorted(io.cleared),
    successful_clear_count=len(io.cleared),total_virtual_s=io.virtual_time,
    average_per_cleared_s=io.virtual_time/len(io.cleared) if io.cleared else None,
    clearance_ratio=None,true_source_count=None,program_elapsed_s=time.monotonic()-io.
    enter_started if io.enter_started else None,ended_by_exit=io.ended,unresolved_action=io.
    uncertain,accepted_actions=io.accepted_actions,http_attempts=io.attempted_requests,
    time_components_s=io.parts,case_code=args.case_code)
212     (run_dir/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),
    encoding='utf-8')
213     if policy is not None:(run_dir/'localization_regions.json').write_text(json.dumps(
    policy.certificates,ensure_ascii=False,indent=2),encoding='utf-8')
214     io.record('summary',**summary);io.log.close()
215     if code==0:print(f'任务完成, 清除 {len(io.cleared)} 个, 平均 {summary["average_per_cleared_s
    "]:.2f} 虚拟秒/源。',flush=True)
216     print(f'结果已保存: {run_dir / "summary.json"}',flush=True)
217     return code

```

```
218
219 if __name__=='__main__':sys.exit(main())
```

## 4.8 参数配置: config.json

```
1 {
2   "seed": 20260910,
3   "cases_per_question": 40,
4   "region_radius_m": 1800,
5   "receiver_radius_min_m": 1000,
6   "receiver_radius_max_m": 1500,
7   "bearing_bound_deg": 1.005,
8   "optical_grid_m": 25,
9   "second_forward_m": 650,
10  "second_lateral_m": 250,
11  "q3_grid_m": 1000,
12  "q3_survey_layout": "hexagon",
13  "q4_grid_m": 600,
14  "speed_m_s": 5,
15  "measure_s": 5,
16  "switch_s": 1,
17  "clear_success_s": 5,
18  "clear_failure_s": 3,
19  "near_m": 5,
20  "clear_m": 20,
21  "virtual_limit_s": 360000,
22  "q4_survey_layout": "triangular",
23  "q4_triangle_spacing_m": 950,
24  "q3_policy": "mixed",
25  "q3_ring_radius_m": 1150,
26  "q3_second_forward_m": 200,
27  "q3_second_lateral_m": 100,
28  "q4_policy": "mixed_ring",
29  "q4_second_forward_m": 200,
30  "q4_second_lateral_m": 100,
31  "q4_selective_remeasure": true,
32  "q4_wait_for_survey": true,
33  "q4_wait_mode": "inner",
34  "q4_wait_stale_m": 1400,
35  "q4_station_help_m": 950,
36  "q4_help_distance": "polygon",
37  "q4_remeasure_min_sine": 0.3,
38  "q4_remeasure_min_gap_m": 300,
39  "q4_refine_max_m": 150,
40  "q4_shared_scan": true,
41  "q4_strip_cover": true,
```

```

42 "q4_immediate_near": true,
43 "q4_scan_cost_gate": true,
44 "q3_refine_max_m": 300,
45 "q3_refine_offset_m": 60,
46 "q4_station_list": [
47   [
48     0.0,
49     0.0
50   ],
51   [
52     990.0,
53     0.0
54   ],
55   [
56     758.4,
57     636.4
58   ],
59   [
60     171.9,
61     975.0
62   ],
63   [
64     -495.0,
65     857.4
66   ],
67   [
68     -930.3,
69     338.6
70   ],
71   [
72     -930.3,
73     -338.6
74   ],
75   [
76     -495.0,
77     -857.4
78   ],
79   [
80     171.9,
81     -975.0
82   ],
83   [
84     758.4,
85     -636.4
86   ],
87   [
88     1866.0,
89     0.0

```

```

90     ],
91     [
92         1616.0,
93         933.0
94     ],
95     [
96         933.0,
97         1616.0
98     ],
99     [
100         0.0,
101         1866.0
102     ],
103     [
104         -933.0,
105         1616.0
106     ],
107     [
108         -1616.0,
109         933.0
110     ],
111     [
112         -1866.0,
113         0.0
114     ],
115     [
116         -1616.0,
117         -933.0
118     ],
119     [
120         -933.0,
121         -1616.0
122     ],
123     [
124         -0.0,
125         -1866.0
126     ],
127     [
128         933.0,
129         -1616.0
130     ],
131     [
132         1616.0,
133         -933.0
134     ]
135 ]
136 }

```